



**Politecnico
di Torino**



Advanced Machine Learning

A.A. 2023/2024

Tatiana Tommasi

Training Neural Networks: Overview

1. One time setup

activation functions, preprocessing, weight initialization, regularization

2. Training dynamics

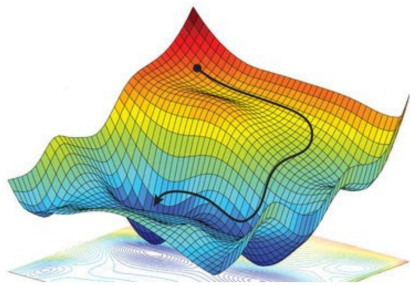
babysitting the learning process, parameter updates, hyperparameter optimization

3. After training

model ensembles, transfer learning

Gradient descent is a **first-order** iterative minimization algorithm.

Example of a loss function $\ell_{\Theta} : \mathbb{R}^2 \rightarrow \mathbb{R}$:



Overall idea: Move where the function decreases the most.

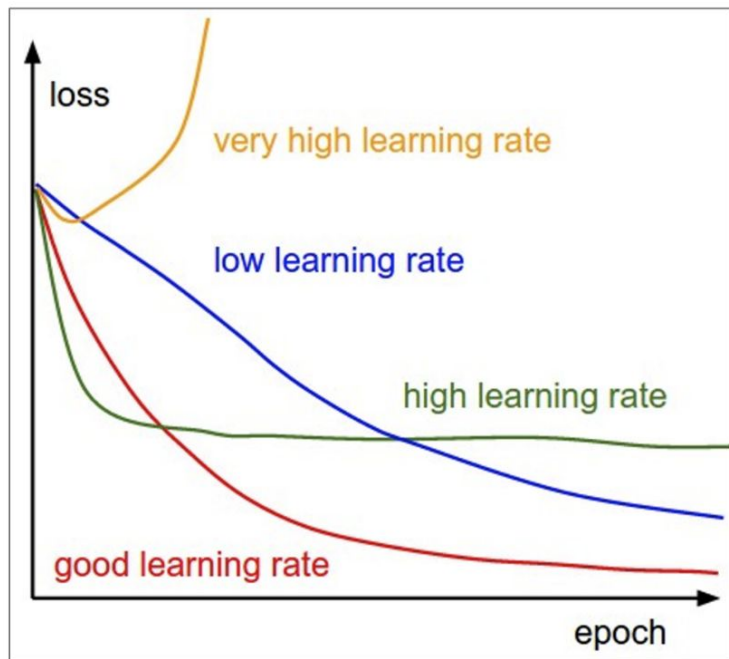
- ① Start from some point $\Theta^{(0)} \in \mathbb{R}^2$.
- ② Iteratively compute:

$$\Theta^{(t+1)} = \Theta^{(t)} - \alpha \nabla \ell_{\Theta^{(t)}}$$

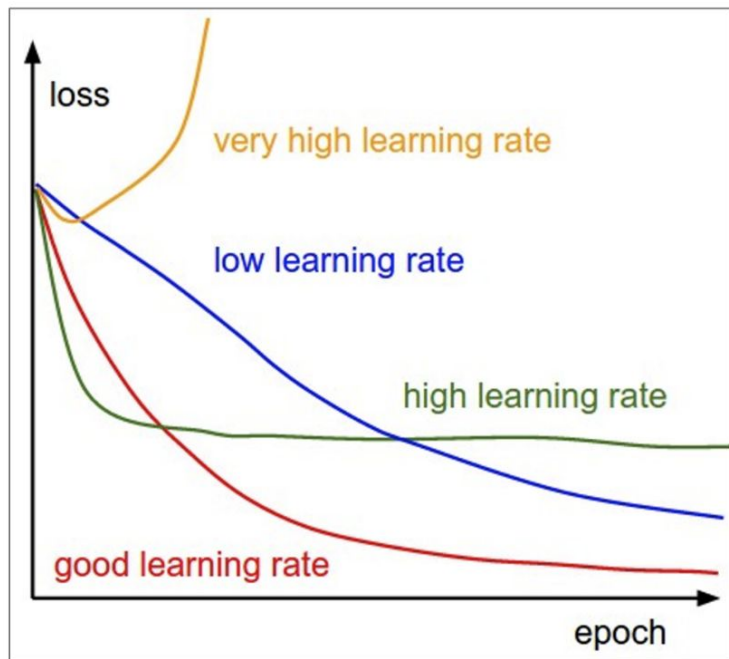
- ③ Stop when a minimum is reached.

Babysitting the learning process [learning rate schedule]

SGD, SGD+Momentum, Adagrad, RMSProp, Adam all have **learning rate** as a hyperparameter.

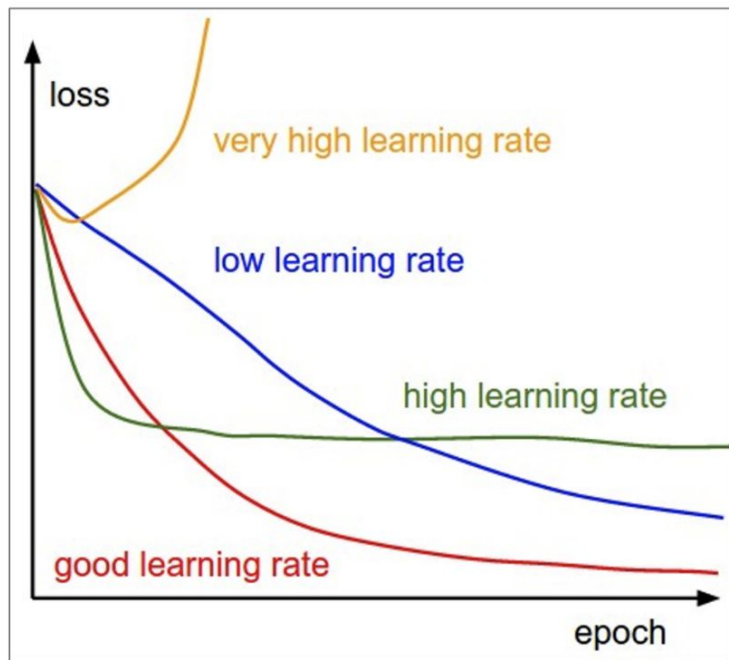


SGD, SGD+Momentum, Adagrad, RMSProp, Adam all have **learning rate** as a hyperparameter.



Q: Which one of these learning rates is best to use?

SGD, SGD+Momentum, Adagrad, RMSProp, Adam all have **learning rate** as a hyperparameter.

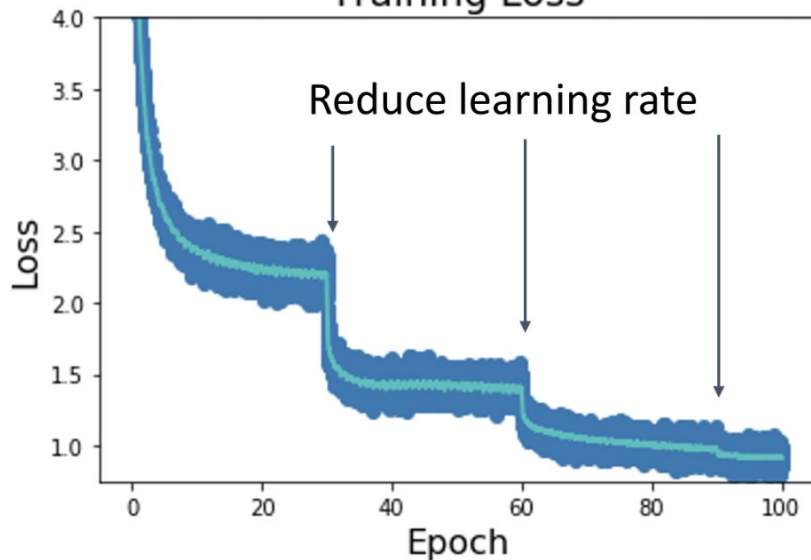


Q: Which one of these learning rates is best to use?

A: All of them! Start with large learning rate and decay over time

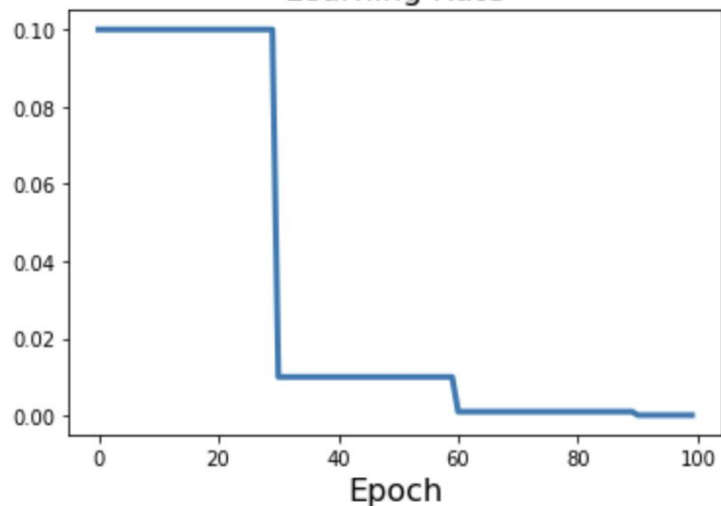
Learning Rate Decay: Step

Training Loss

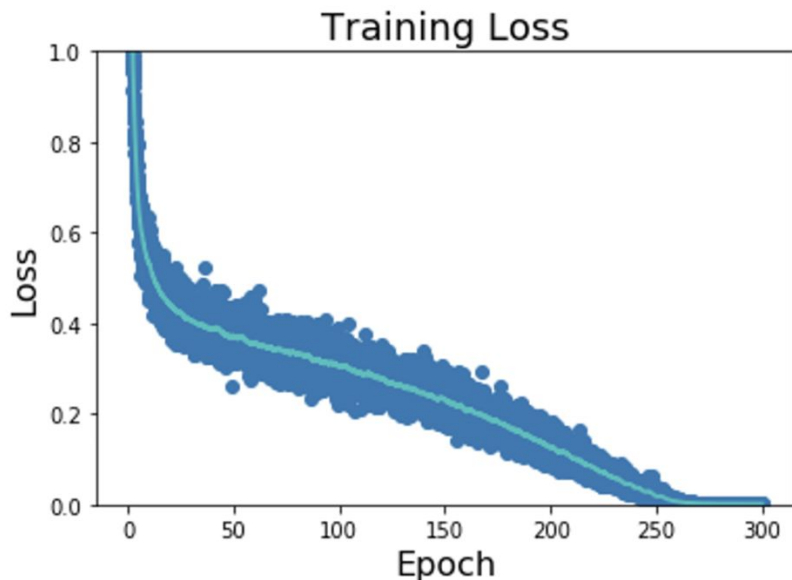


Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Learning Rate

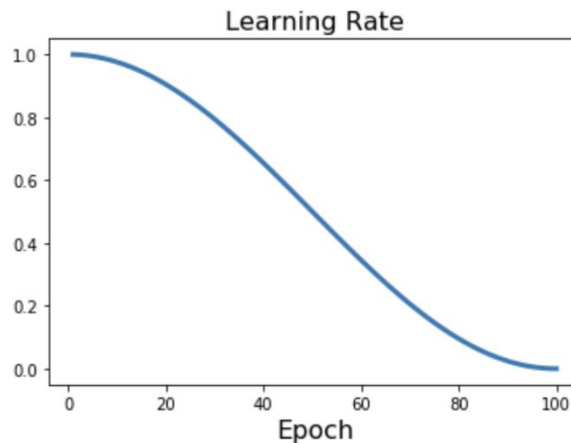


Learning Rate Decay: Cosine



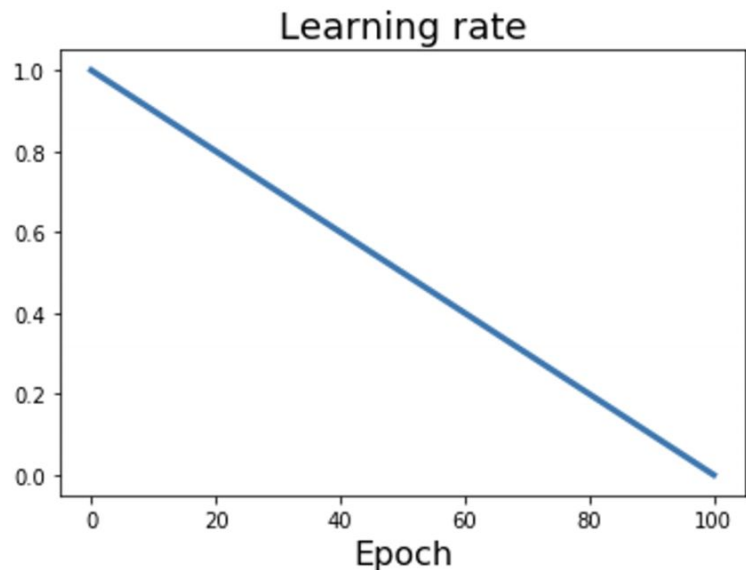
Step: Reduce learning rate at a few fixed points.
E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine:
$$\alpha_t = \frac{1}{2} \alpha_0 \left(1 + \cos \left(\frac{t\pi}{T} \right) \right)$$



Loshchilov and Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts", ICLR 2017
Radford et al, "Improving Language Understanding by Generative Pre-Training", 2018
Feichtenhofer et al, "SlowFast Networks for Video Recognition", ICCV 2019
Radosavovic et al, "On Network Design Spaces for Visual Recognition", ICCV 2019
Child et al, "Generating Long Sequences with Sparse Transformers", arXiv 2019

Learning Rate Decay: Linear



Step: Reduce learning rate at a few fixed points.
E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine:
$$\alpha_t = \frac{1}{2} \alpha_0 \left(1 + \cos \left(\frac{t\pi}{T} \right) \right)$$

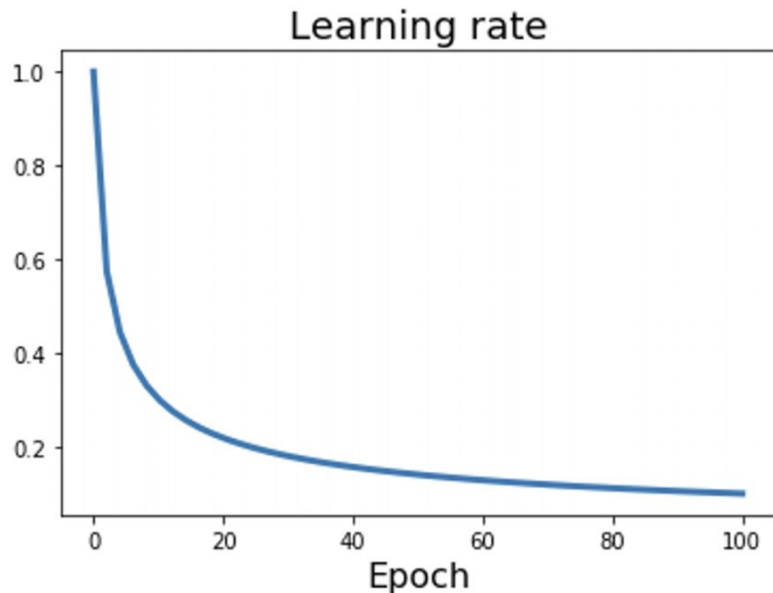
Linear:
$$\alpha_t = \alpha_0 \left(1 - \frac{t}{T} \right)$$

Devlin et al, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", NAACL 2018

Liu et al, "RoBERTa: A Robustly Optimized BERT Pretraining Approach", 2019

Yang et al, "XLNet: Generalized Autoregressive Pretraining for Language Understanding", NeurIPS 2019

Learning Rate Decay: Inverse Sqrt



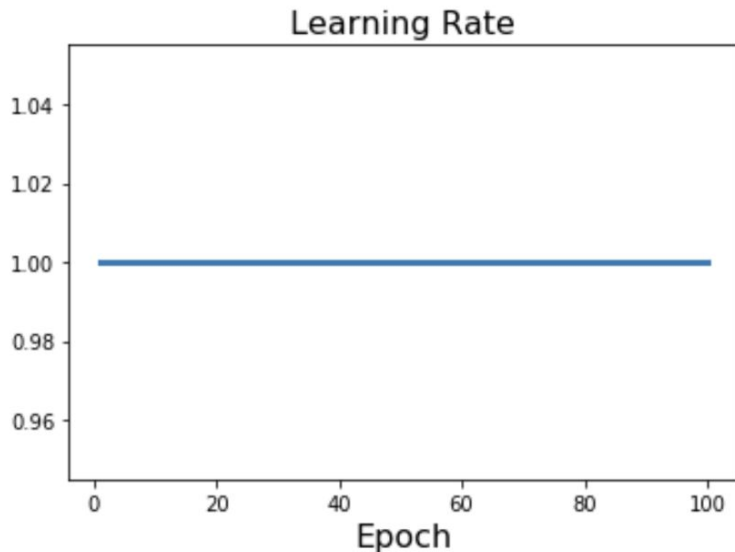
Step: Reduce learning rate at a few fixed points.
E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2} \alpha_0 \left(1 + \cos \left(\frac{t\pi}{T} \right) \right)$

Linear: $\alpha_t = \alpha_0 \left(1 - \frac{t}{T} \right)$

Inverse sqrt: $\alpha_t = \alpha_0 / \sqrt{t}$

Learning Rate Decay: Constant!



Step: Reduce learning rate at a few fixed points.
E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2} \alpha_0 \left(1 + \cos \left(\frac{t\pi}{T} \right) \right)$

Linear: $\alpha_t = \alpha_0 \left(1 - \frac{t}{T} \right)$

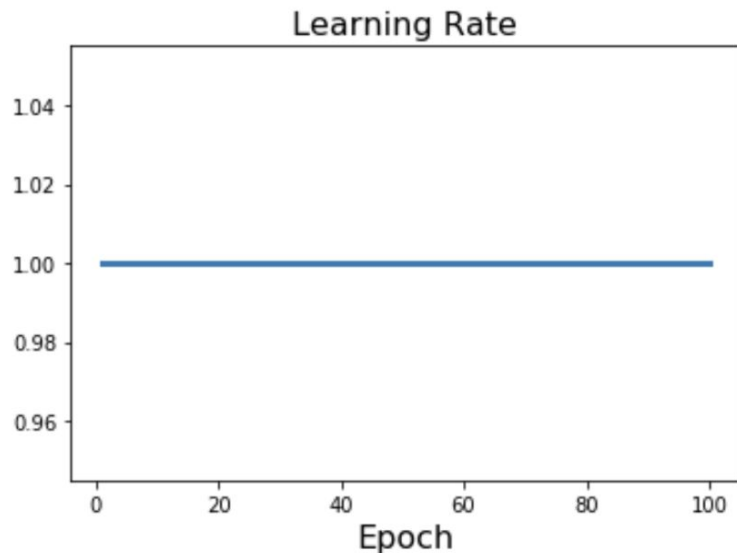
Inverse sqrt: $\alpha_t = \alpha_0 / \sqrt{t}$

Constant: $\alpha_t = \alpha_0$

Brock et al, "Large Scale GAN Training for High Fidelity Natural Image Synthesis", ICLR 2019

Donahue and Simonyan, "Large Scale Adversarial Representation Learning", NeurIPS 2019

Learning Rate Decay: Constant!



Step: Reduce learning rate at a few fixed points.
E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2} \alpha_0 \left(1 + \cos \left(\frac{t\pi}{T} \right) \right)$

Linear: $\alpha_t = \alpha_0 \left(1 - \frac{t}{T} \right)$

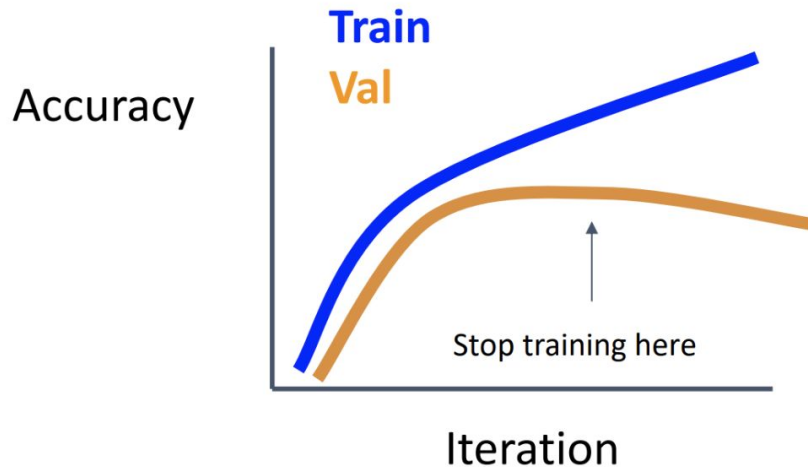
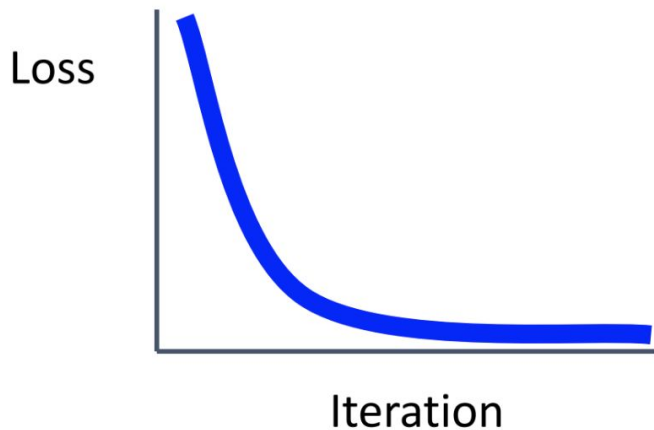
Inverse sqrt: $\alpha_t = \alpha_0 / \sqrt{t}$

Constant: $\alpha_t = \alpha_0$

Brock et al, "Large Scale GAN Training for High Fidelity Natural Image Synthesis", ICLR 2019

Donahue and Simonyan, "Large Scale Adversarial Representation Learning", NeurIPS 2019

How long to train? Early Stopping



Stop training the model when accuracy on the validation set decreases
Or train for a long time, but always keep track of the model snapshot that worked best on val. **Always a good idea to do this!**

Hyperparameter Optimization

Choosing Hyperparameters: Grid Search

Choose several values for each hyperparameter
(Often space choices log-linearly)

Example:

Weight decay: $[1 \times 10^{-4}, 1 \times 10^{-3}, 1 \times 10^{-2}, 1 \times 10^{-1}]$

Learning rate: $[1 \times 10^{-4}, 1 \times 10^{-3}, 1 \times 10^{-2}, 1 \times 10^{-1}]$

Evaluate all possible choices on this
hyperparameter grid

Choosing Hyperparameters: Random Search

Choose several values for each hyperparameter
(Often space choices log-linearly)

Example:

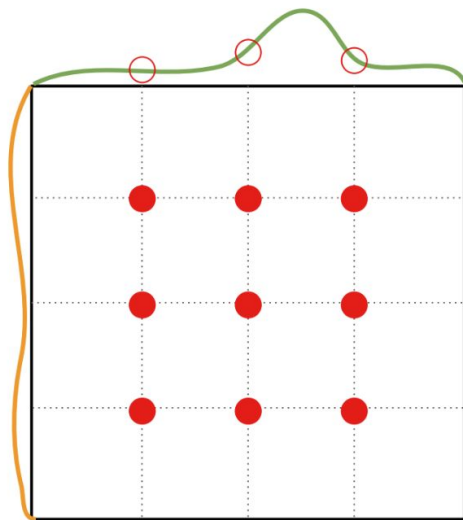
Weight decay: log-uniform on $[1 \times 10^{-4}, 1 \times 10^{-1}]$

Learning rate: log-uniform on $[1 \times 10^{-4}, 1 \times 10^{-1}]$

Run many different trials

Hyperparameters: Random vs Grid Search

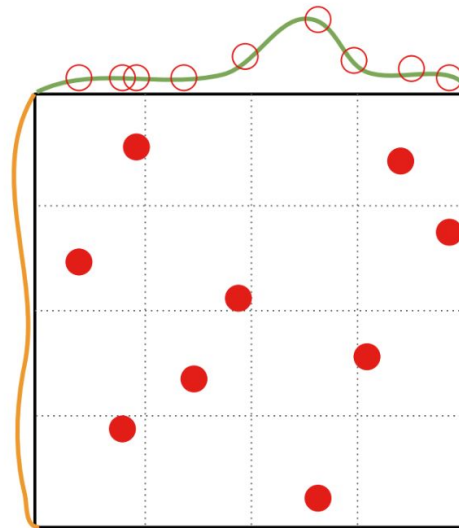
Grid Layout



Unimportant
Parameter

Important
Parameter

Random Layout

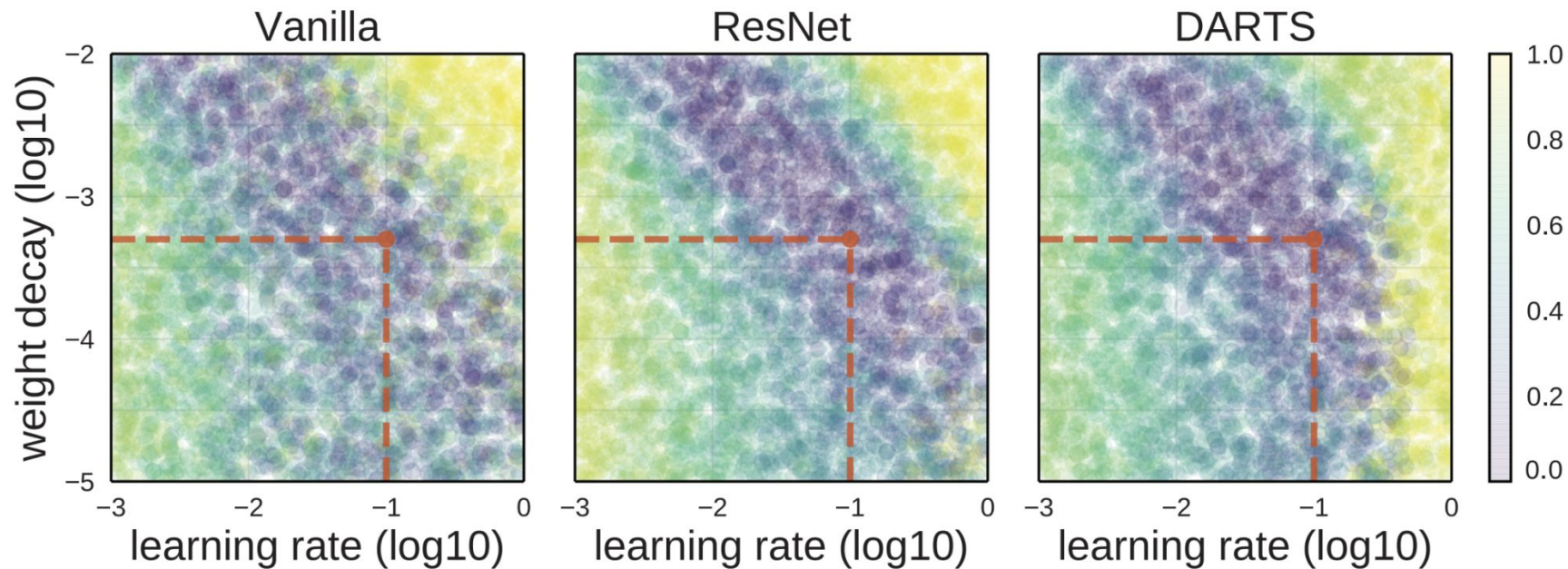


Unimportant
Parameter

Important
Parameter

Bergstra and Bengio, "Random Search for Hyper-Parameter Optimization", JMLR 2012

Choosing Hyperparameters: Random Search



Radosavovic et al, "On Network Design Spaces for Visual Recognition", ICCV 2019

Choosing Hyperparameters

Step 1: Check initial loss

Turn off weight decay, sanity check loss at initialization
e.g. $\log(C)$ for softmax with C classes

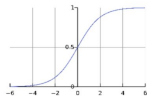
Logistic Regression

New loss:

$$\ell_{\Theta}(\{x_i, y_i\}) = -\sum_{i=1}^n y_i \ln(\sigma(ax_i + b)) + (1 - y_i) \ln(1 - \sigma(ax_i + b))$$

Here, σ is the nonlinear **logistic sigmoid**:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

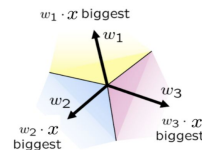


σ has a **saturation** effect as it maps $\mathbb{R} \mapsto (0, 1)$.

Multiclass Logistic Regression

Recall Perceptron:

- A weight vector for each class: w_y
- Score (activation) of a class y : $w_y \cdot \mathcal{X}$
- Prediction highest score wins $y = \arg \max_y w_y \cdot \mathcal{X}$

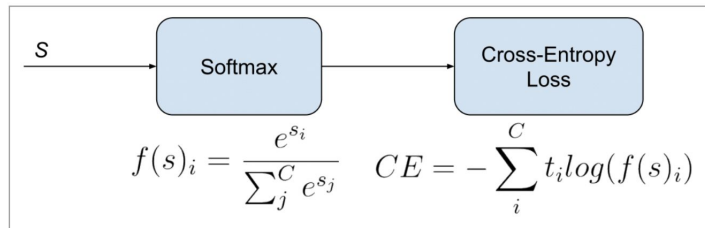


How to make the scores into probabilities?

$$\underbrace{z_1, z_2, z_3}_{\text{original activations}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}}_{\text{softmax activations}}$$

Categorical Cross-Entropy loss

Also called **Softmax Loss**. It is a **Softmax activation** plus a **Cross-Entropy loss**. If we use this loss, we will train a CNN to output a probability over the C classes for each image. It is used for multi-class classification.



In the specific (and usual) case of Multi-Class classification the labels are one-hot, so only the positive class C_p keeps its term in the loss. There is only one element of the Target vector t which is not zero $t_i = t_p$. So discarding the elements of the summation which are zero due to target labels, we can write:

$$CE = -\log \left(\frac{e^{s_p}}{\sum_j^C e^{s_j}} \right)$$

Choosing Hyperparameters

Step 1: Check initial loss

Step 2: Overfit a small sample

Try to train to 100% training accuracy on a small sample of training data (~5-10 minibatches); fiddle with architecture, learning rate, weight initialization. Turn off regularization.

Loss not going down? LR too low, bad initialization

Loss explodes to Inf or NaN? LR too high, bad initialization

Choosing Hyperparameters

Step 1: Check initial loss

Step 2: Overfit a small sample

Step 3: Find LR that makes loss go down

Use the architecture from the previous step, use all training data, turn on small weight decay, find a learning rate that makes the loss drop significantly within ~ 100 iterations

Good learning rates to try: $1e-1$, $1e-2$, $1e-3$, $1e-4$

Choosing Hyperparameters

Step 1: Check initial loss

Step 2: Overfit a small sample

Step 3: Find LR that makes loss go down

Step 4: Coarse grid, train for ~1-5 epochs

Choose a few values of learning rate and weight decay around what worked from Step 3, train a few models for ~1-5 epochs.

Good weight decay to try: $1e-4$, $1e-5$, 0

Choosing Hyperparameters

Step 1: Check initial loss

Step 2: Overfit a small sample

Step 3: Find LR that makes loss go down

Step 4: Coarse grid, train for $\sim 1-5$ epochs

Step 5: Refine grid, train longer

Pick best models from Step 4, train them for longer
($\sim 10-20$ epochs) without learning rate decay

Choosing Hyperparameters

Step 1: Check initial loss

Step 2: Overfit a small sample

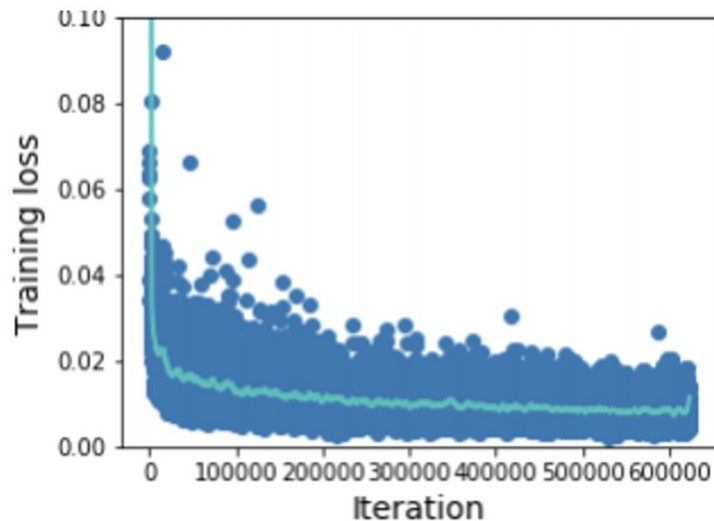
Step 3: Find LR that makes loss go down

Step 4: Coarse grid, train for ~1-5 epochs

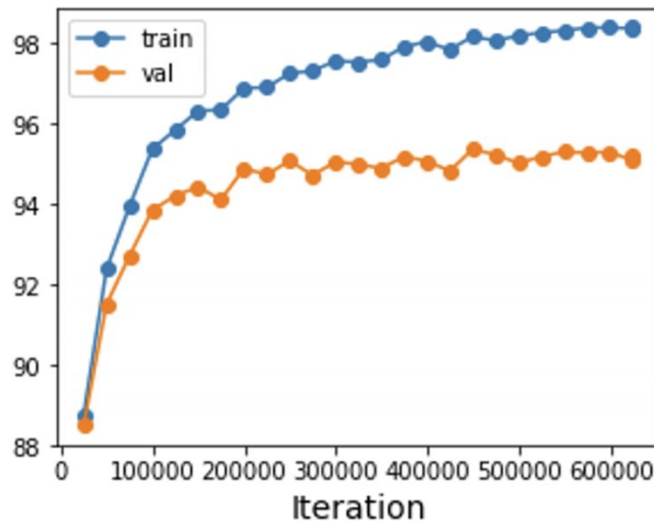
Step 5: Refine grid, train longer

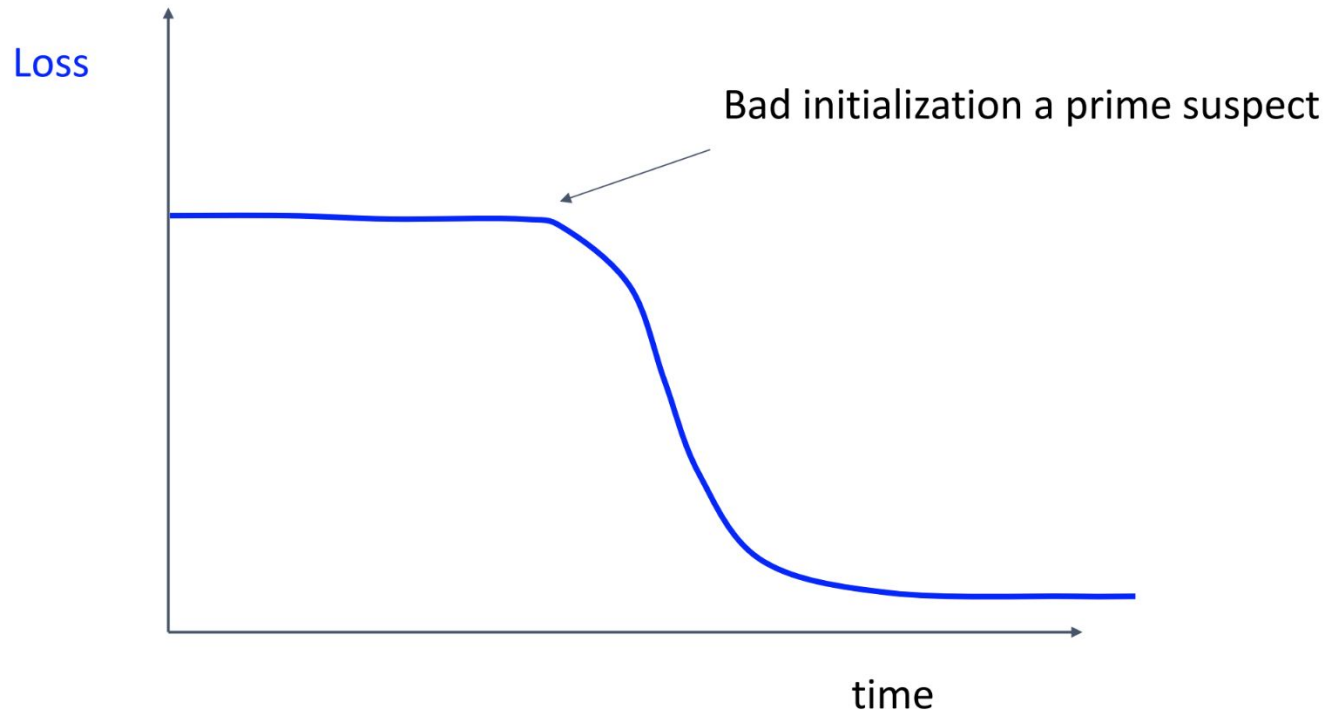
Step 6: Look at learning curves

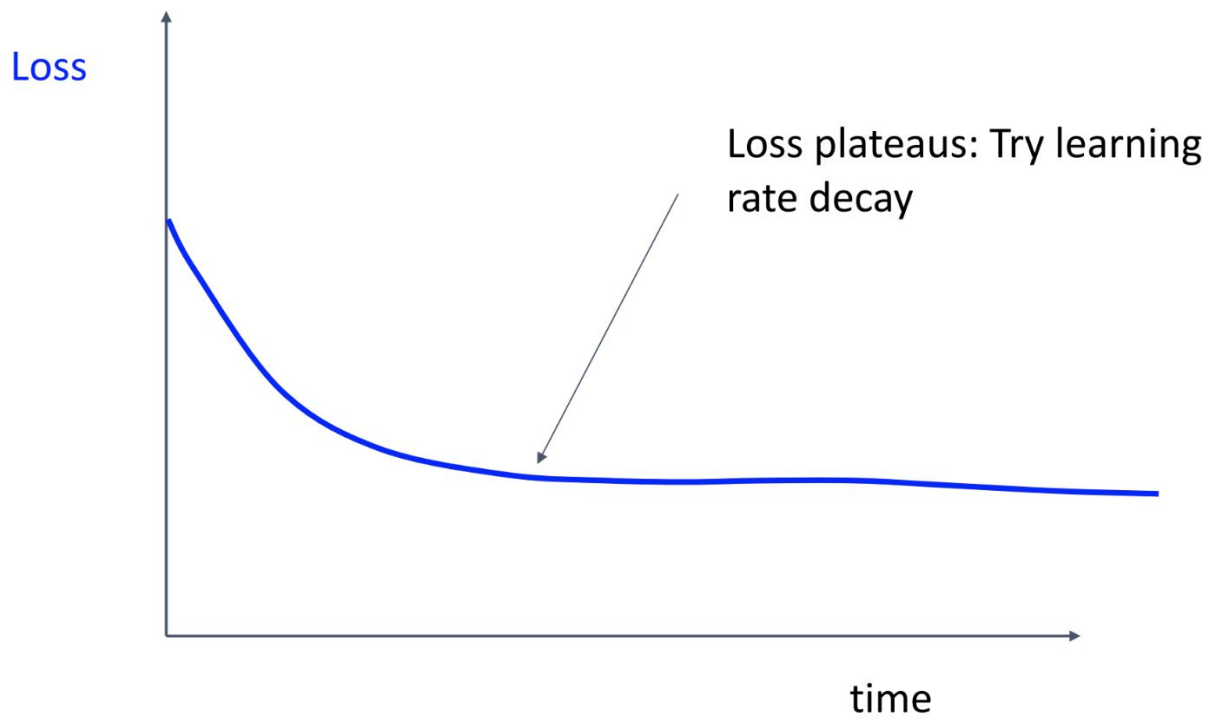
Look at Learning Curves!

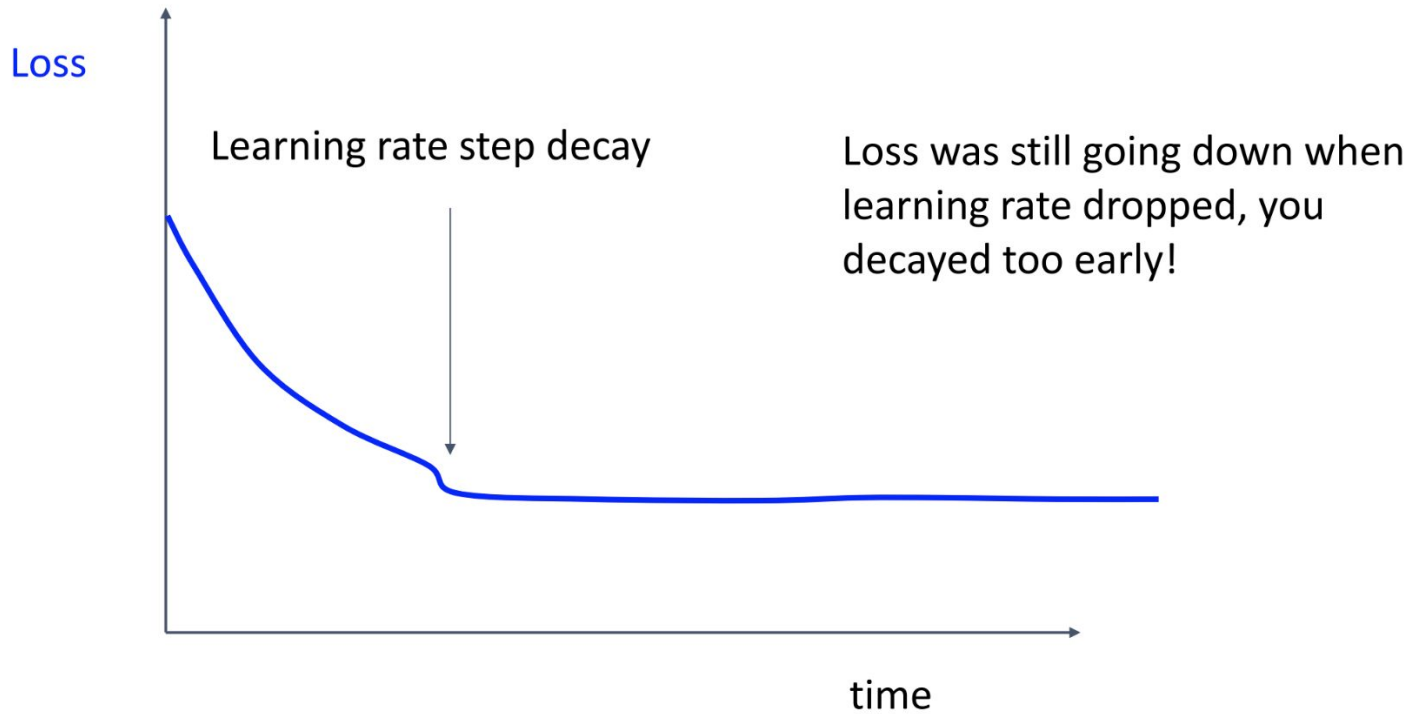


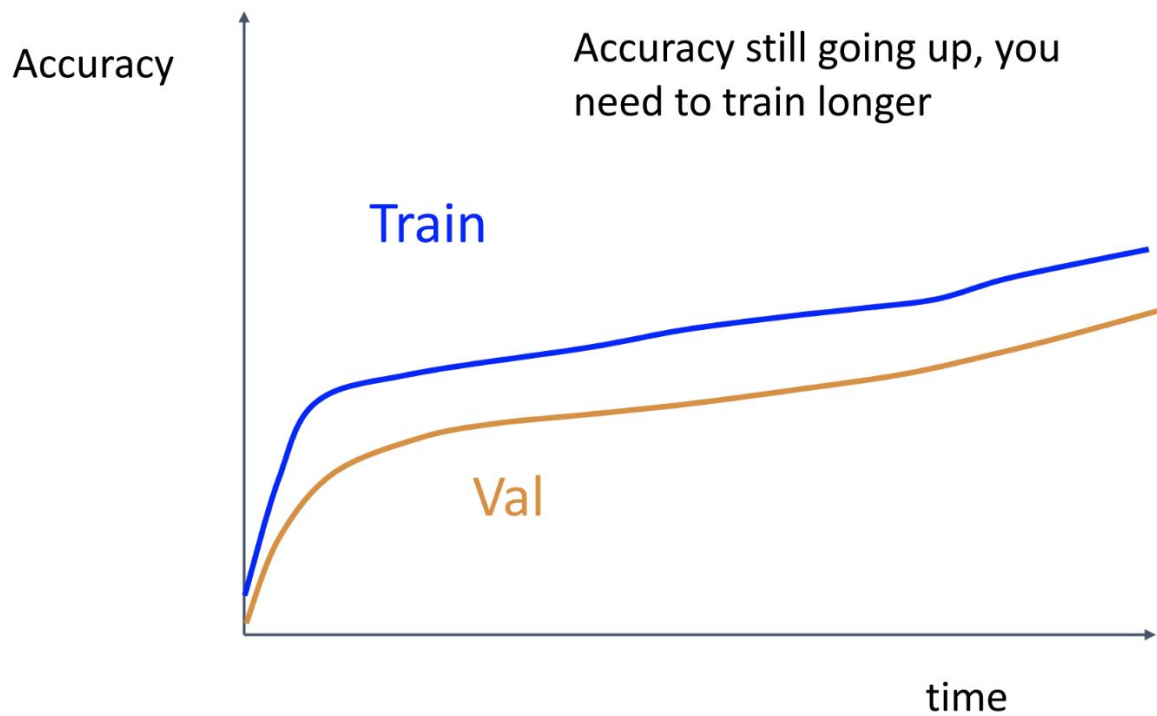
Losses may be noisy, use a scatter plot and also plot moving average to see trends better

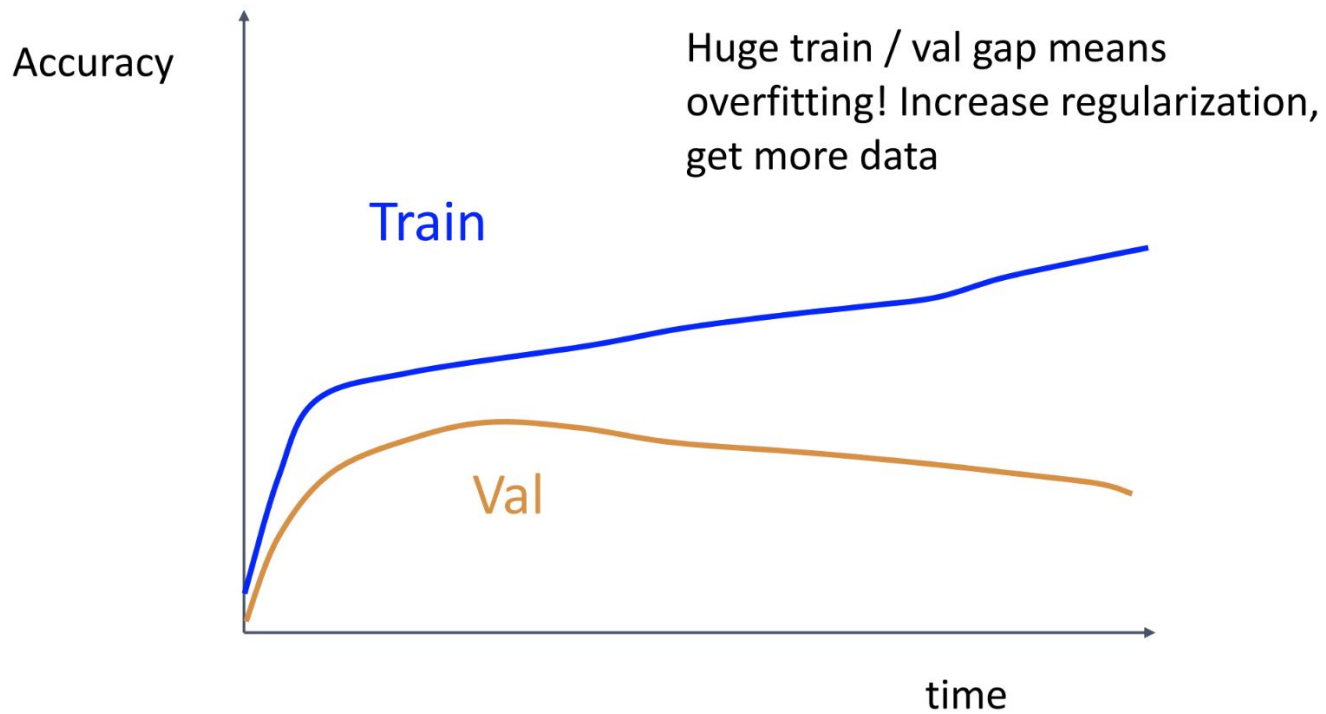












Accuracy

No gap between train / val means
underfitting: train longer, use a bigger
model

Train

Val

time

Choosing Hyperparameters

Step 1: Check initial loss

Step 2: Overfit a small sample

Step 3: Find LR that makes loss go down

Step 4: Coarse grid, train for $\sim 1-5$ epochs

Step 5: Refine grid, train longer

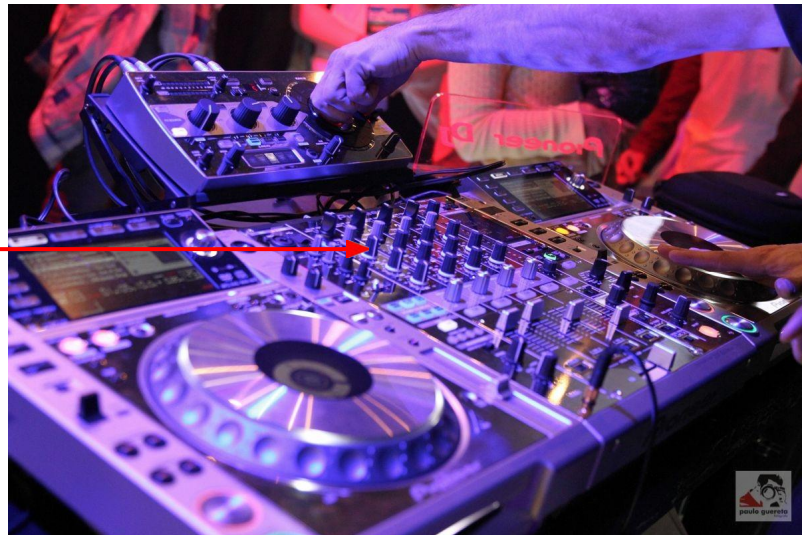
Step 6: Look at loss curves

Step 7: GOTO step 5

Hyperparameters to play with:

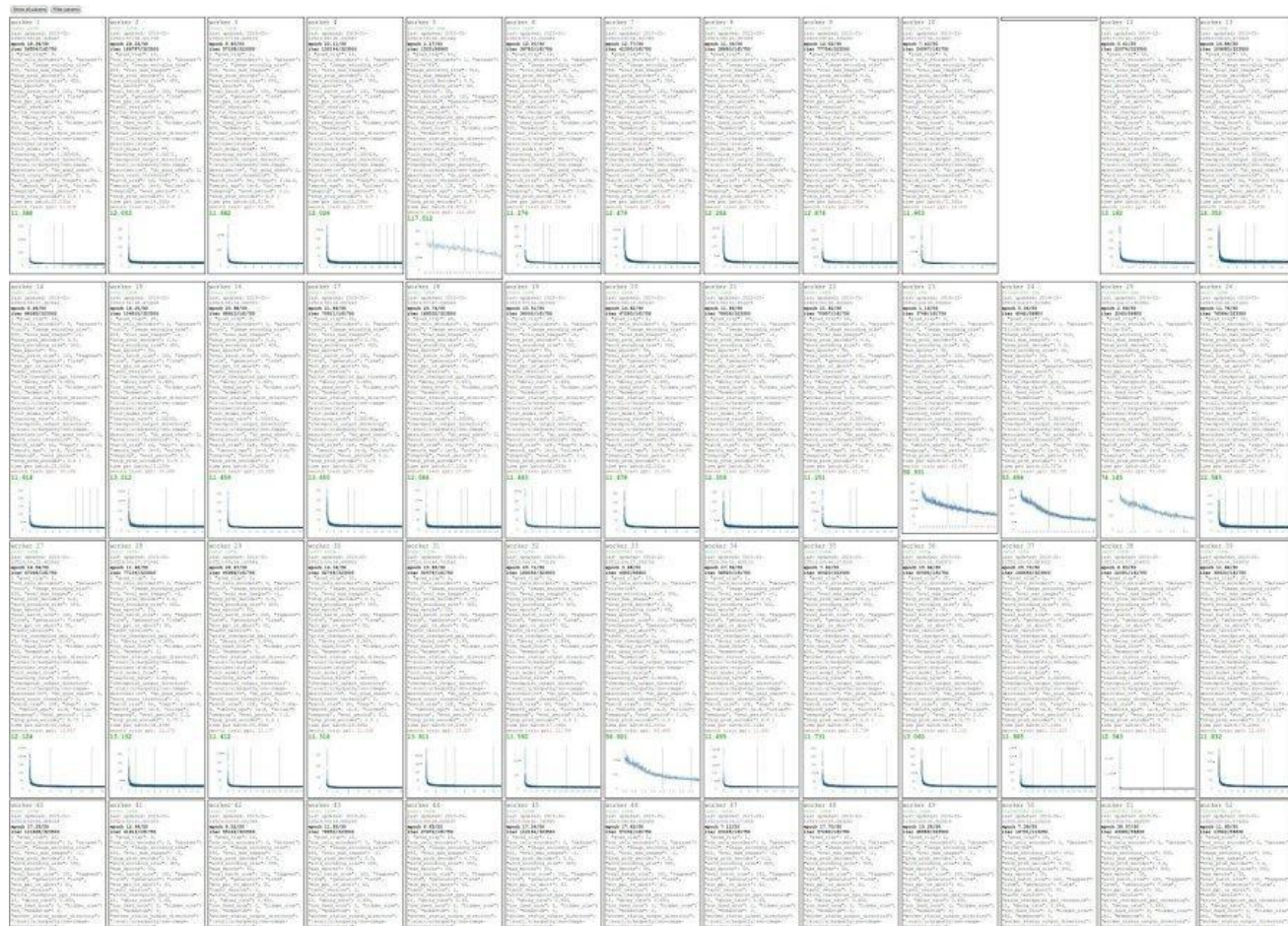
- network architecture
- learning rate, its decay schedule, update type
- regularization (L2/Dropout strength)

neural networks practitioner
music = loss function



[This image](#) by Paolo Guereta is licensed under [CC-BY 2.0](#)

Cross-validation “command center”



Track ratio of weight update / weight magnitude

```
# assume parameter vector W and its gradient vector dW
param_scale = np.linalg.norm(W.ravel())
update = -learning_rate*dW # simple SGD update
update_scale = np.linalg.norm(update.ravel())
W += update # the actual update
print update_scale / param_scale # want ~1e-3
```

ratio between the updates and values: $\sim 0.0002 / 0.02 = 0.01$ (about okay)
want this to be somewhere around 0.001 or so

Summary

We looked in detail at:

- Activation Functions (use ReLU)
- Data Preprocessing (images: subtract mean)
- Weight Initialization (use Xavier init)
- Batch Normalization (use)
- Babysitting the Learning process
- Hyperparameter Optimization
(random sample hyperparams, in log space when appropriate)

Training Neural Networks: Overview

1. One time setup

activation functions, preprocessing, weight initialization, regularization

2. Training dynamics

babysitting the learning process, parameter updates, hyperparameter optimization

3. After training

*model ensembles, **transfer learning***

Transfer Learning

“You need a lot of a data if you want to train/use CNNs”

Transfer Learning

BUSTED
“You need a lot of data if you want to
train/use CNNs”

<http://www.image-net.org/>

ImageNet is an image database organized according to the **WordNet** hierarchy (currently only the nouns), in which each node of the hierarchy is depicted by hundreds and thousands of images. Currently we have an average of over five hundred images per node. We hope ImageNet will become a useful resource for researchers, educators, students and all of you who share our passion for pictures.

[Click here](#) to learn more about ImageNet, [Click here](#) to join the ImageNet mailing list.



What do these images have in common? *Find out!*

Research updates on improving ImageNet data

ImageNet Tree View x Johnny

www.image-net.org/synset?wnid=n13003061

IMAGENET 14,197,122 images, 21841 synsets indexed

Home About Explore Download

Not logged in. Login | Signup

Fly agaric, *Amanita muscaria*

Poisonous (but rarely fatal) woodland fungus having a scarlet cap with white warts and white gills

1444 pictures 29.59% Popularity Percentile Wordnet IDs

Numbers in brackets: (the number of synsets in the subtree).

- ImageNet 2011 Fall Release (3232)
 - plant, flora, plant life (4486)
 - geological formation, formation
 - natural object (1112)
 - sport, athletics (176)
 - artifact, artefact (10504)
 - fungus (308)
 - false morel (6)
 - pythium (1)
 - truffle, earthnut, earth-ball (Candida)
 - candida (1)
 - shiitake, shiitake mushroom, shiitake (0)
 - sac fungus (0)
 - lichen (11)
 - brown root rot fungus, Thielavia (0)
 - dry rot (0)
 - white fungus, Saprolegnia ferax (5)
 - mildew (5)
 - earthball, false truffle, puffball (2)
 - yeast (2)
 - bird's-nest fungus (0)
 - green smut fungus, Ustilagium (0)
 - hen-of-the-woods, hen of the woods (0)
 - verticillium (0)
 - scaly lentinus, Lentinus lepideus (0)
 - monilia (0)

[Treemap Visualization](#)
[Images of the Synset](#)
[Downloads](#)

Transfer Learning with CNN

1. Train on Imagenet

FC-1000
FC-4096
FC-4096

MaxPool
Conv-512
Conv-512

MaxPool
Conv-512
Conv-512

MaxPool
Conv-256
Conv-256

MaxPool
Conv-128
Conv-128

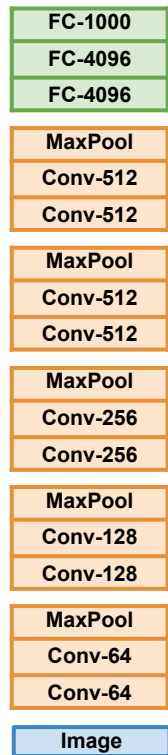
MaxPool
Conv-64
Conv-64

Image

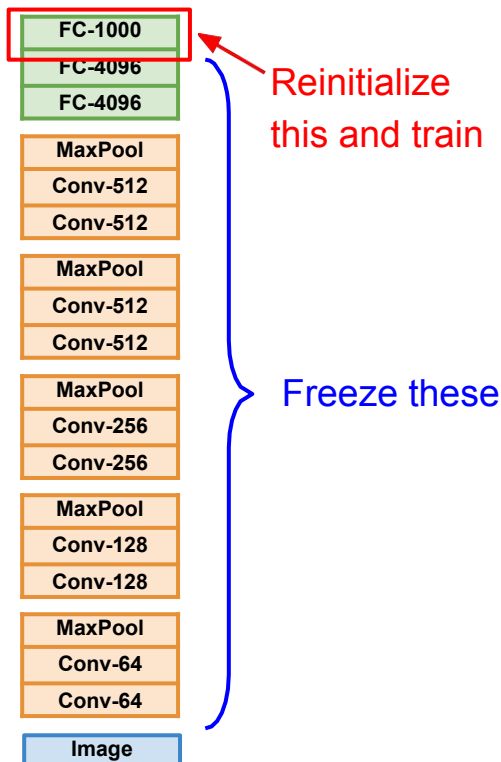
Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

Transfer Learning with CNN

1. Train on Imagenet

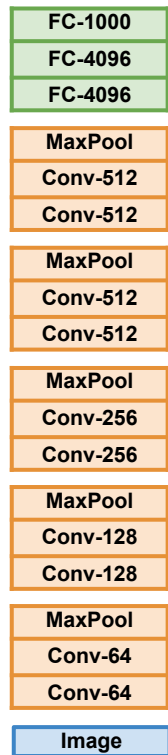


2. Small Dataset (C classes)

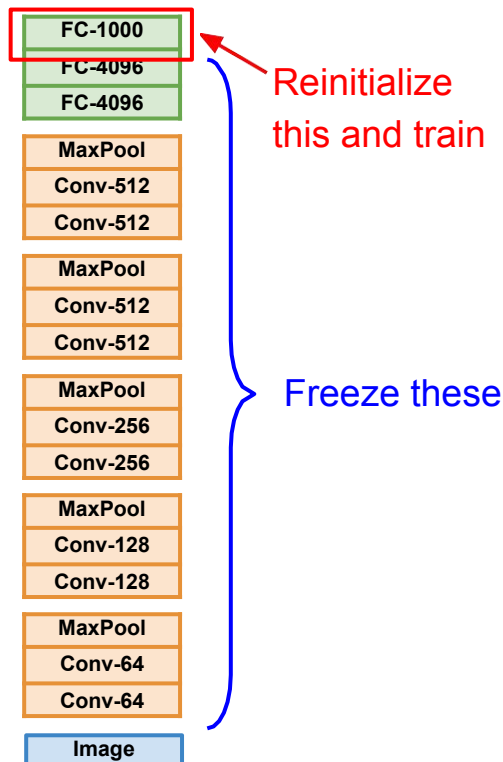


Transfer Learning with CNN

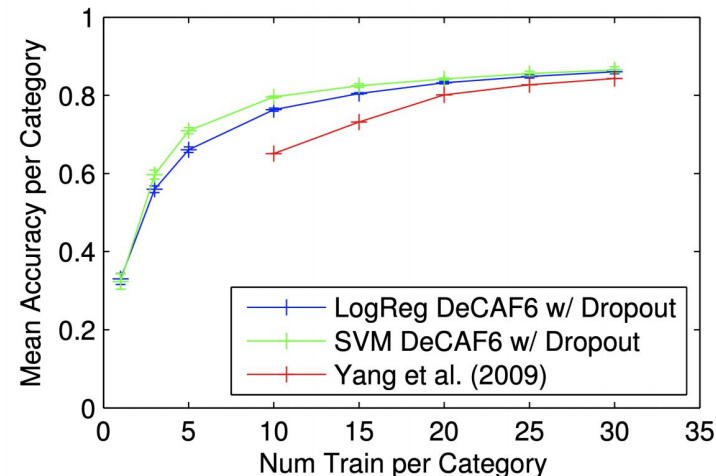
1. Train on Imagenet



2. Small Dataset (C classes)

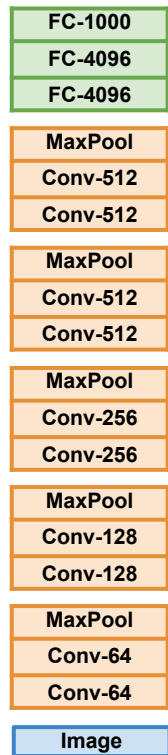


Classification on Caltech-101

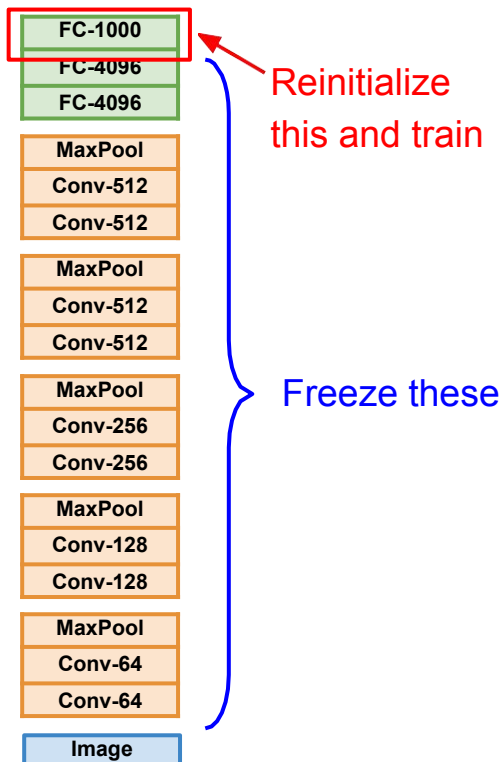


Transfer Learning with CNN

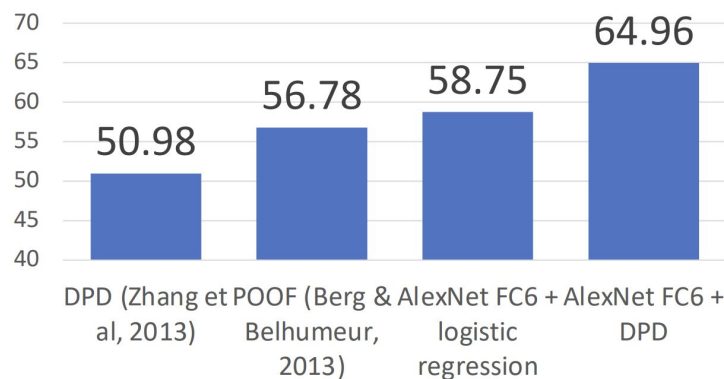
1. Train on Imagenet



2. Small Dataset (C classes)

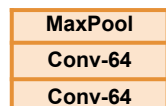
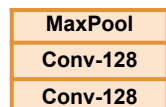
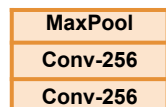
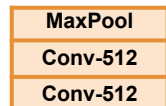
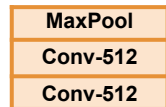
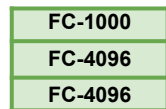


Bird Classification on Caltech-UCSD

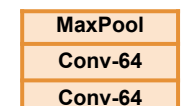
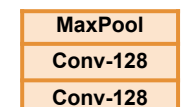
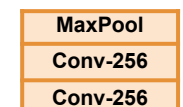
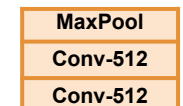
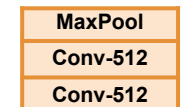
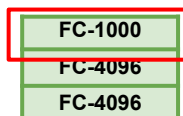


Transfer Learning with CNN

1. Train on Imagenet



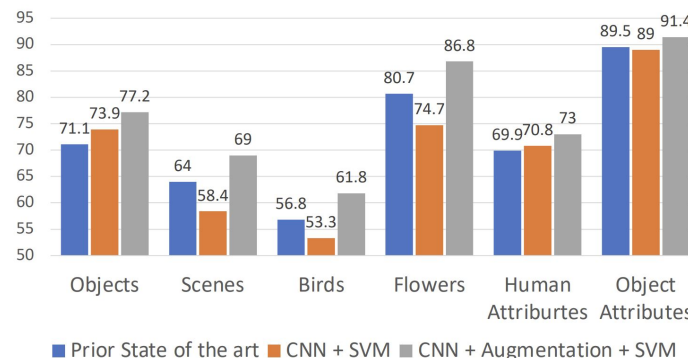
2. Small Dataset (C classes)



Reinitialize
this and train

Freeze these

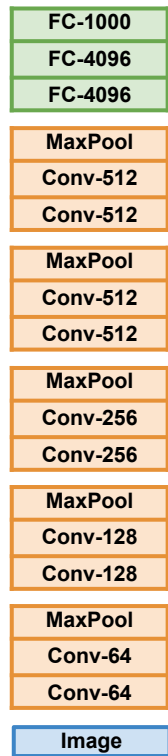
Image Classification



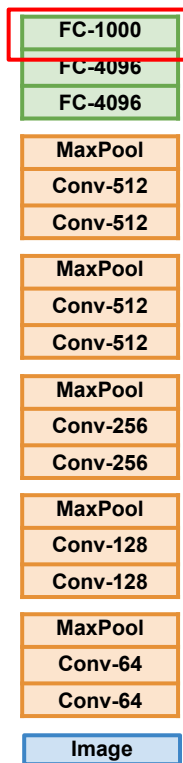
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

Transfer Learning with CNN

1. Train on Imagenet



2. Small Dataset (C classes)

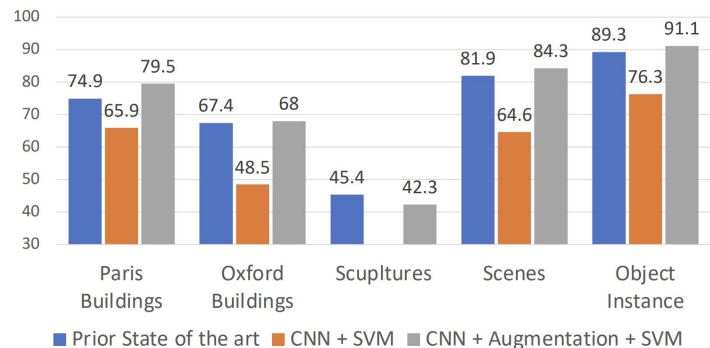


Reinitialize
this and train

Freeze these

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

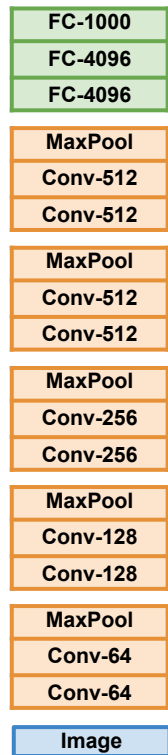
Image Retrieval: Nearest-Neighbor



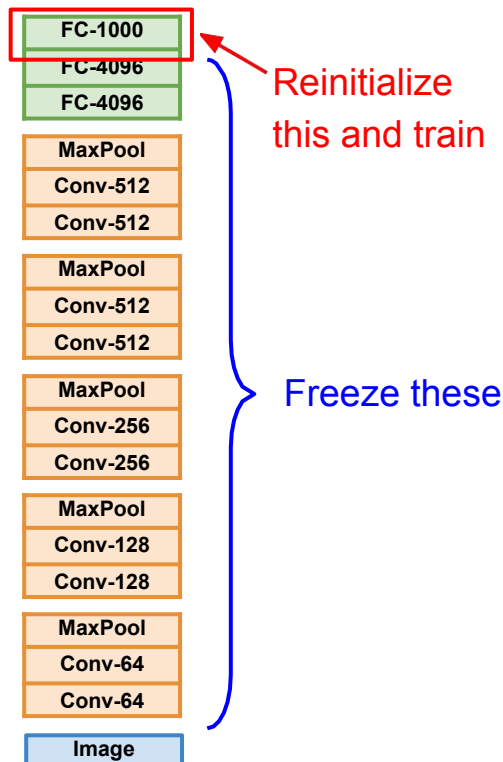
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

Transfer Learning with CNN

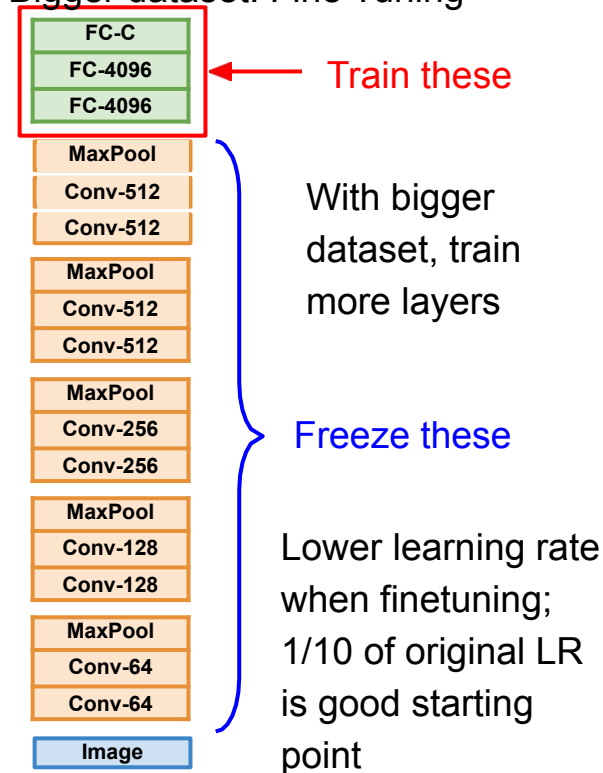
1. Train on Imagenet



2. Small Dataset (C classes)

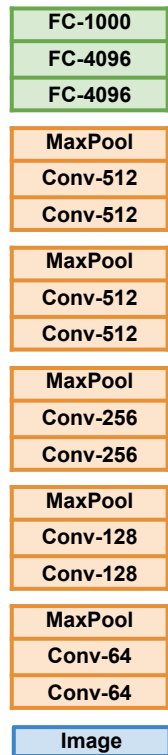


3. Bigger dataset: Fine Tuning

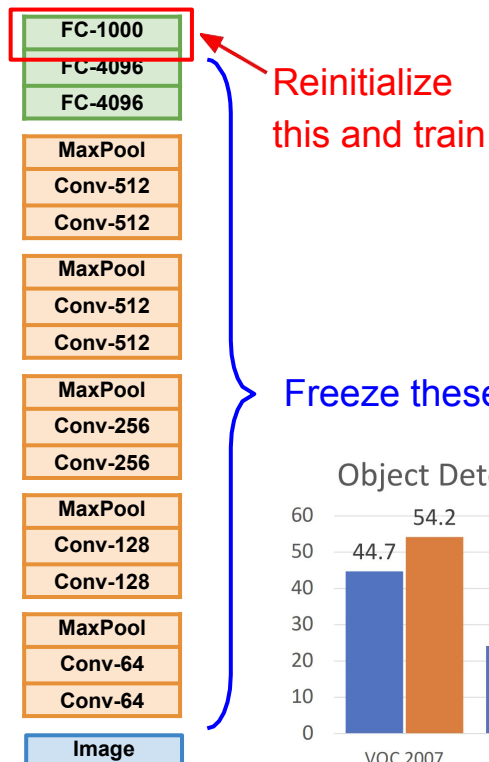


Transfer Learning with CNN

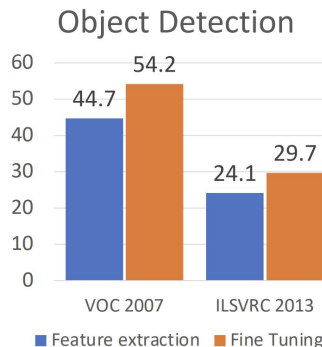
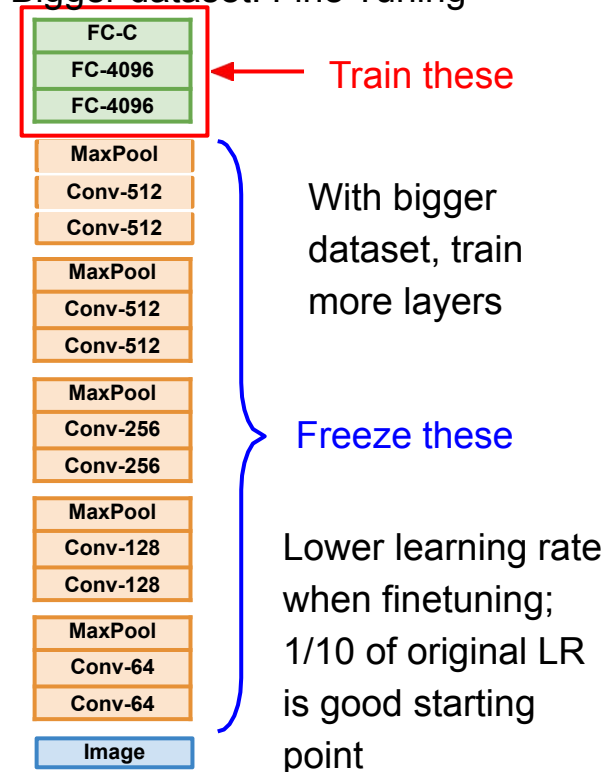
1. Train on Imagenet



2. Small Dataset (C classes)

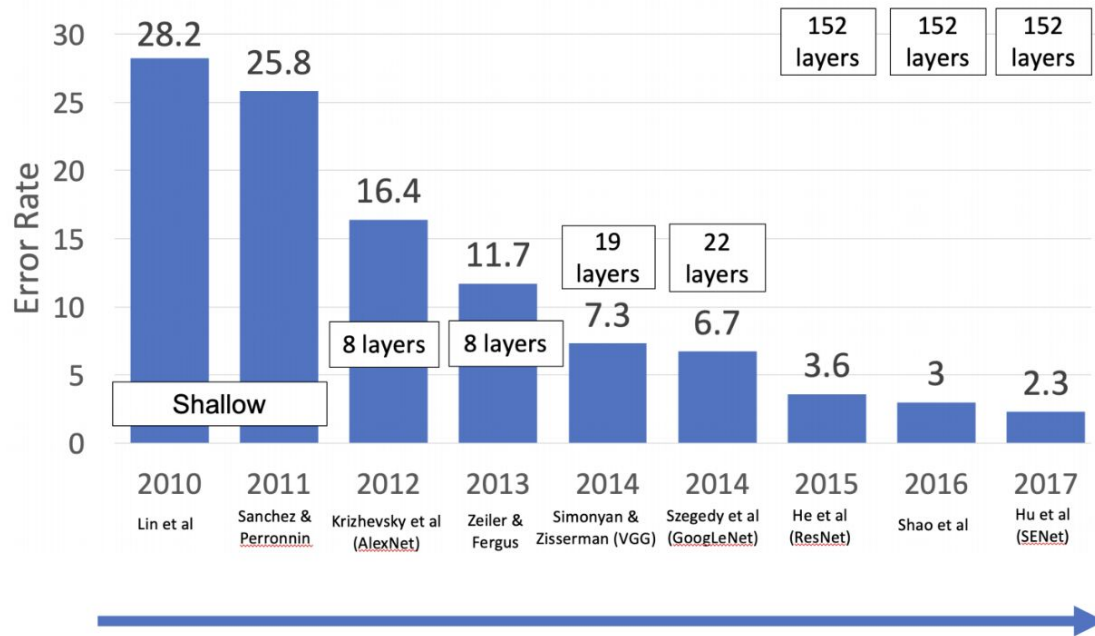


3. Bigger dataset: Fine Tuning



Transfer Learning with CNNs: Architecture Matters!

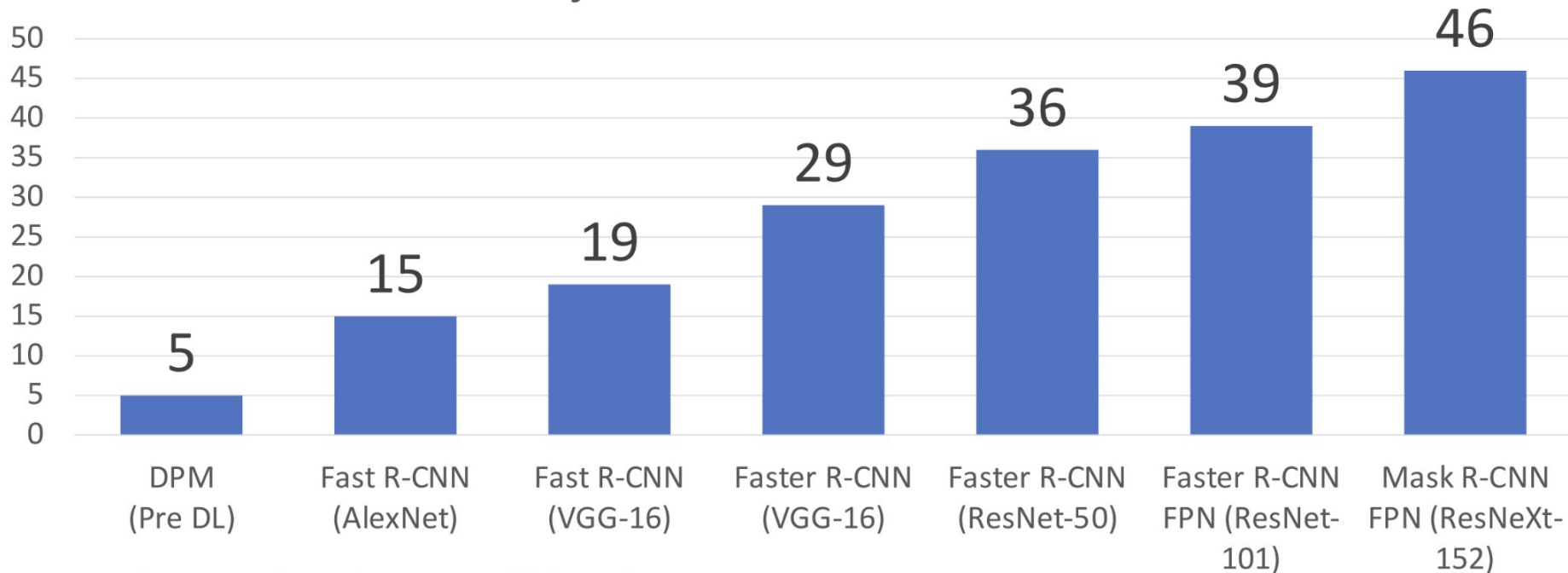
ImageNet Classification Challenge



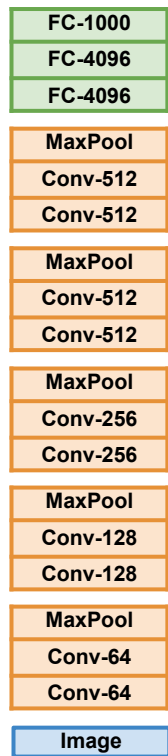
Improvements in CNN architectures lead to improvements in many downstream tasks thanks to transfer learning!

Transfer Learning with CNNs: Architecture Matters!

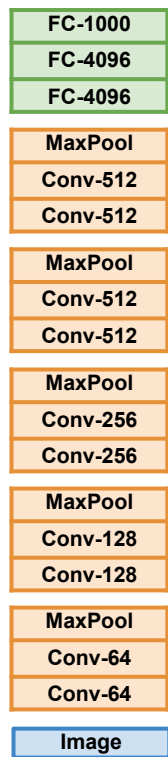
Object Detection on COCO



Ross Girshick, "The Generalized R-CNN Framework for Object Detection", ICCV 2017 Tutorial on Instance-Level Visual Recognition



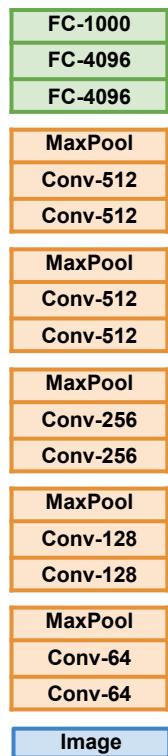
	very similar dataset	very different dataset
very little data (10-100 samples per class)	?	?
quite a lot of data (100-1000 samples per class)	?	?



More specific

More generic

	very similar dataset	very different dataset
very little data (10-100 samples per class)	Use Linear Classifier on top layer	?
quite a lot of data (100-1000 samples per class)	Finetune a few layers	?



More specific

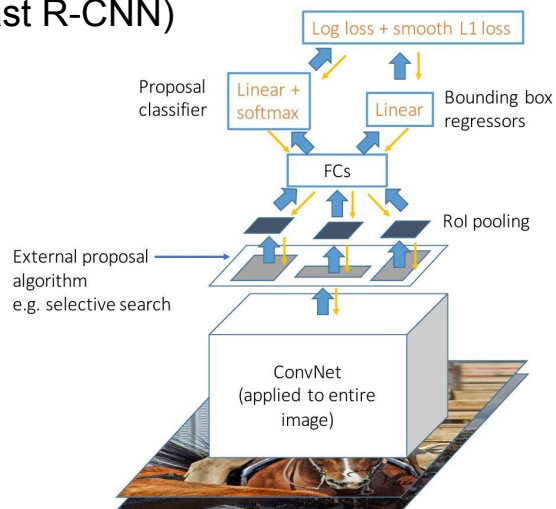
More generic

	very similar dataset	very different dataset
very little data (10-100 samples per class)	Use Linear Classifier on top layer	You're in trouble... Try linear classifier from different stages
quite a lot of data (100-1000 samples per class)	Finetune a few layers	Finetune a larger number of layers

Transfer Learning with CNN is pervasive...

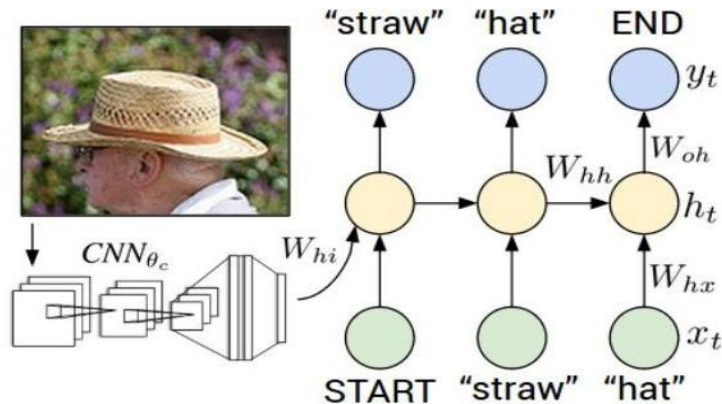
(it's the norm, not an exception)

Object Detection (Fast R-CNN)



Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Image Captioning: CNN + RNN

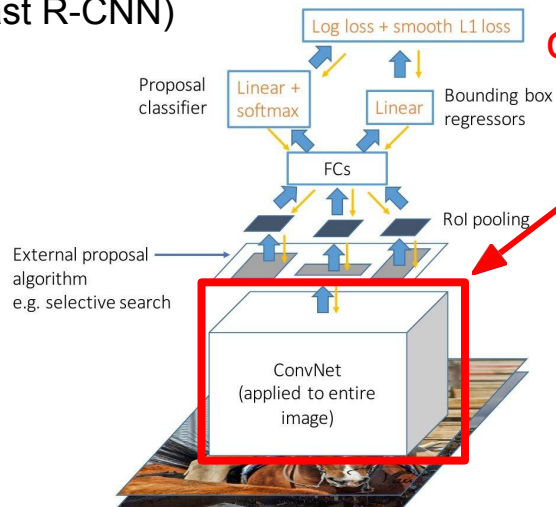


Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

Transfer Learning with CNN is pervasive...

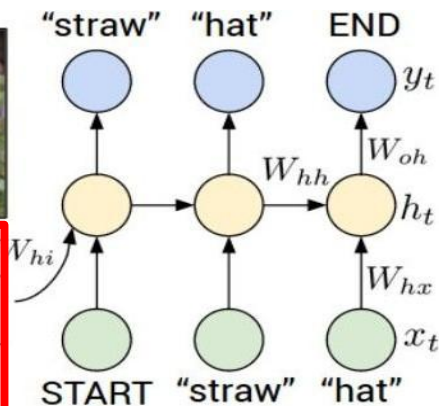
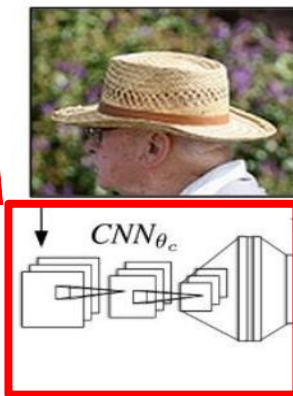
(it's the norm, not an exception)

Object Detection (Fast R-CNN)



CNN pretrained
on ImageNet

Image Captioning: CNN + RNN



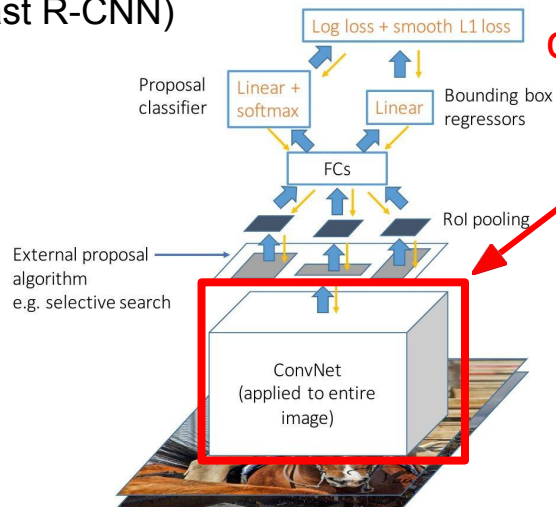
Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for
Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

Transfer Learning with CNN is pervasive...

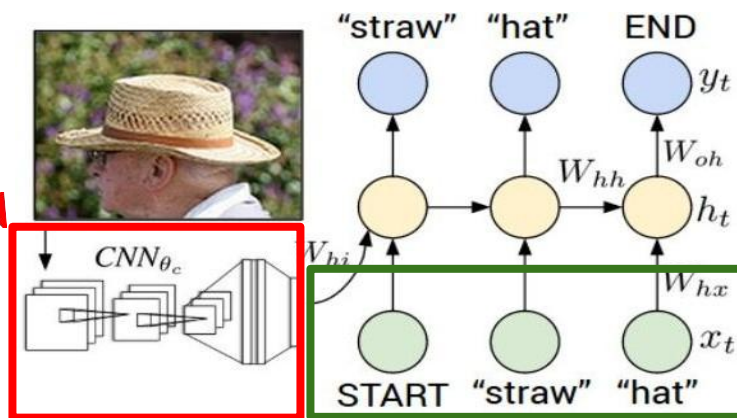
(it's the norm, not an exception)

Object Detection (Fast R-CNN)



CNN pretrained
on ImageNet

Image Captioning: CNN + RNN



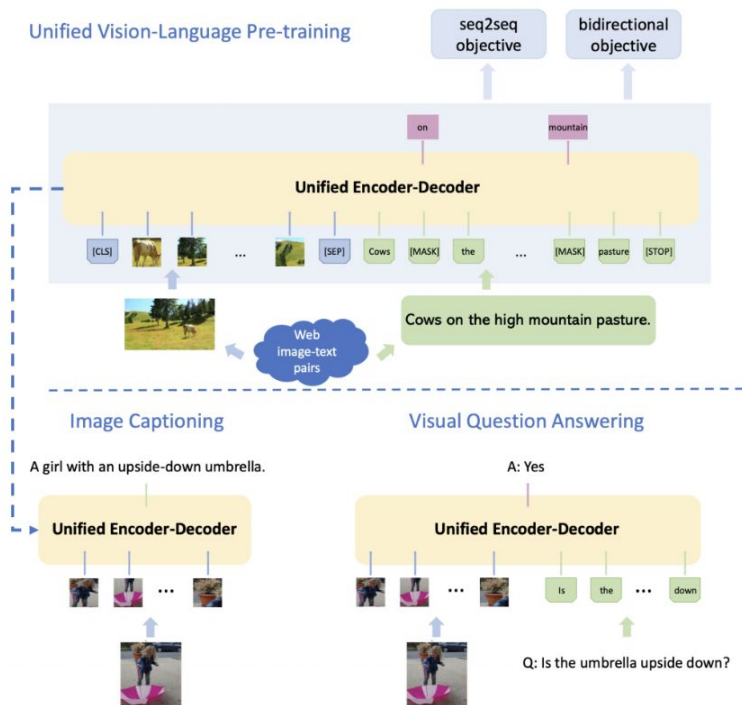
Word vectors pre-trained
with word2vec

Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for
Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

Transfer learning is pervasive!

It's the norm, not the exception



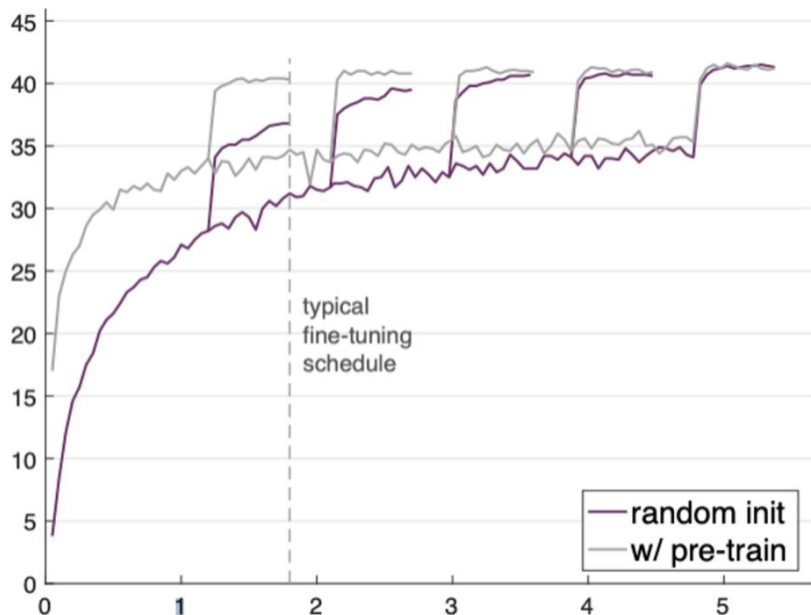
1. Train CNN on ImageNet
2. Fine-Tune (1) for object detection on Visual Genome
3. Train BERT language model on lots of text
4. Combine (2) and (3), train for joint image / language modeling
5. Fine-tune (5) for image captioning, visual question answering, etc.

Zhou et al, "Unified Vision-Language Pre-Training for Image Captioning and VQA", arXiv 2019

Transfer learning is pervasive!

Some very recent results have questioned it

COCO object detection



Training from scratch can work as well as pretraining on ImageNet!

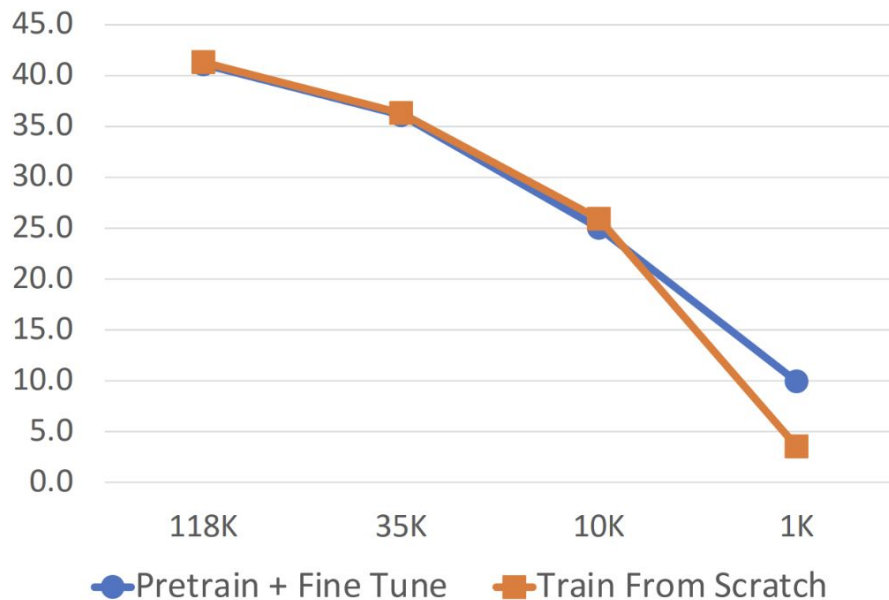
... If you train for 3x as long

He et al, "Rethinking ImageNet Pre-Training", ICCV 2019

Transfer learning is pervasive!

Some very recent results have questioned it

COCO object detection



Pretraining + Finetuning beats training from scratch when dataset size is very small

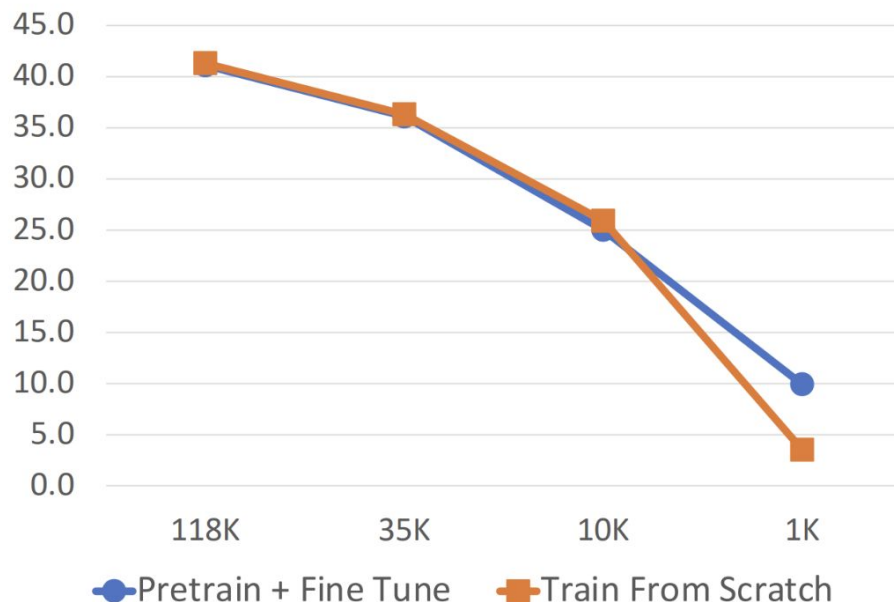
Collecting more data is more effective than pretraining

He et al, "Rethinking ImageNet Pre-Training", ICCV 2019

Transfer learning is pervasive!

Some very recent results have questioned it

COCO object detection



My current view on transfer learning:

- Pretrain+finetune makes your training faster, so practically very useful
- Training from scratch works well once you have enough data
- Lots of work left to be done

He et al, "Rethinking ImageNet Pre-Training", ICCV 2019

And now we move to the Architectures

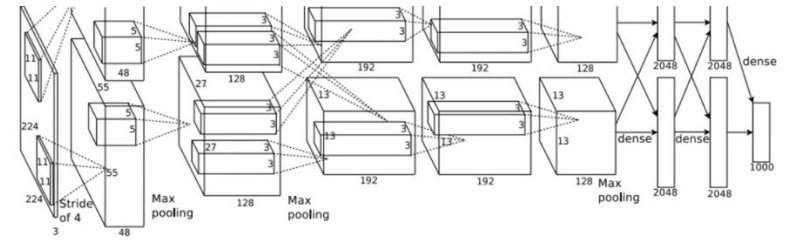
Case Studies

- **AlexNet**
- VGG
- GoogLeNet
- ResNet

The diagram illustrates the architecture of the proposed deep learning model. It starts with an input volume of size 224x224x3. This is followed by three stages of Max pooling (stride of 4), which reduces the spatial dimensions while maintaining the depth. The resulting volumes are then processed by four dense layers. The first dense layer has 2048 units, followed by two more dense layers each with 2048 units, and finally a classification layer with 1000 units. The output is a segmented volume.

[illegible]

AlexNet

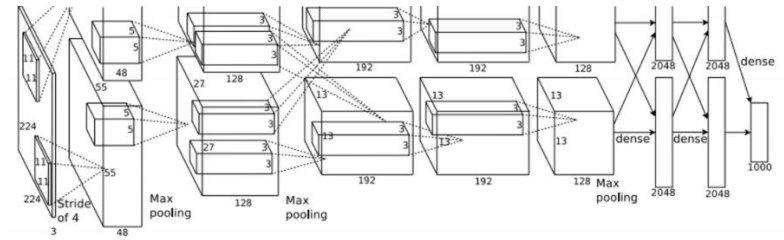


Layer	Input size		Layer				Output size	
	C	H / W	filters	kernel	stride	pad	C	H / W
conv1	3	227	64	11	4	2	64	?

Recall: Output channels = number of filters

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet

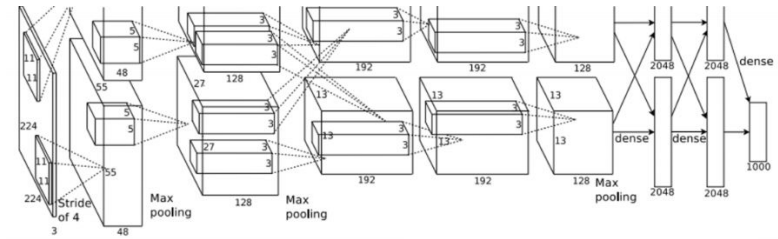


Layer	Input size		Layer				Output size	
	C	H / W	filters	kernel	stride	pad	C	H / W
conv1	3	227	64	11	4	2	64	56

$$\begin{aligned}
 \text{Recall: } W' &= (W - K + 2P) / S + 1 \\
 &= (227 - 11 + 2 \cdot 2) / 4 + 1 \\
 &= 220 / 4 + 1 = 56
 \end{aligned}$$

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

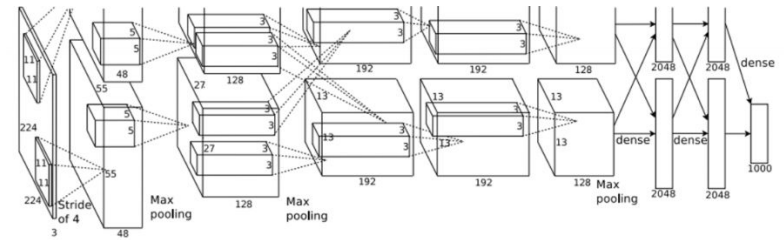
AlexNet



Layer	Input size		Layer				Output size		
	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)
conv1	3	227	64	11	4	2	64	56	?

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



Input size		Layer						Output size		
Layer	C	H / W	filters	kernel	stride	pad		C	H / W	memory (KB)
conv1	3	227	64	11	4	2		64	56	784

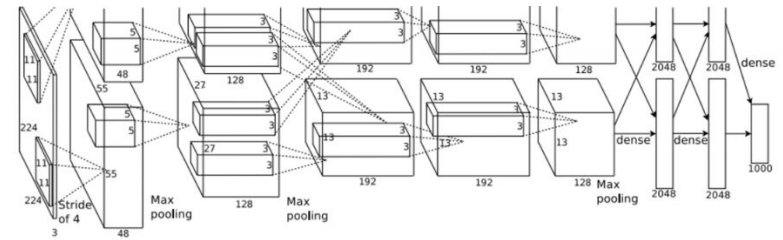
$$\begin{aligned}\text{Number of output elements} &= C * H' * W' \\ &= 64 * 56 * 56 = 200,704\end{aligned}$$

$$\text{Bytes per element} = 4 \text{ (for 32-bit floating point)}$$

$$\begin{aligned}\text{KB} &= (\text{number of elements}) * (\text{bytes per elem}) / 1024 \\ &= 200704 * 4 / 1024 \\ &= \mathbf{784}\end{aligned}$$

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

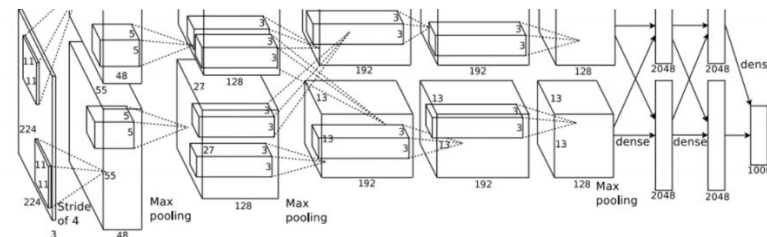
AlexNet



	Input size		Layer				Output size			
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)
conv1		3 227	64	11	4	2	64	56	784	?

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



Layer	Input size		Layer				Output size		memory (KB)	params (k)
	C	H / W	filters	kernel	stride	pad	C	H / W		
conv1	3	227	64	11	4	2	64	56	784	23

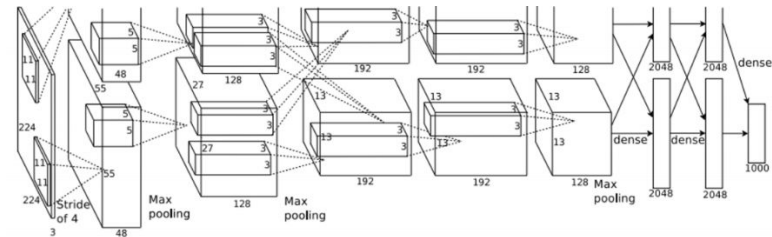
$$\begin{aligned}\text{Weight shape} &= C_{\text{out}} \times C_{\text{in}} \times K \times K \\ &= 64 \times 3 \times 11 \times 11\end{aligned}$$

$$\text{Bias shape} = C_{\text{out}} = 64$$

$$\begin{aligned}\text{Number of weights} &= 64 \times 3 \times 11 \times 11 + 64 \\ &= \mathbf{23,296}\end{aligned}$$

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

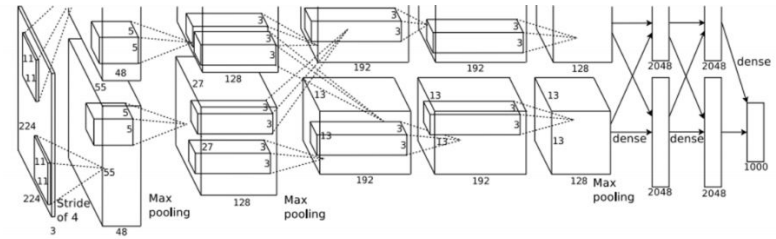
AlexNet



Layer	Input size		Layer				Output size				
	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	?

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet

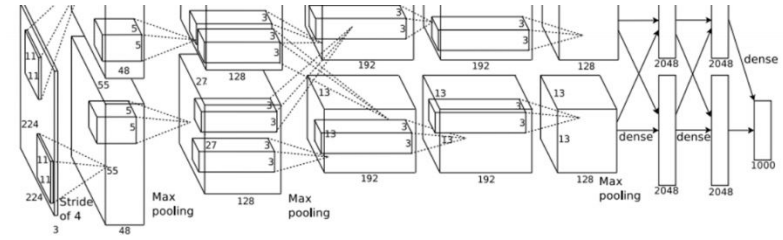


Input size		Layer					Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73

Number of floating point operations (multiply+add)
 = (number of output elements) * (ops per output elem)
 = $(C_{out} \times H' \times W') * (C_{in} \times K \times K)$
 = $(64 * 56 * 56) * (3 * 11 * 11)$
 = $200,704 * 363$
 = **72,855,552**

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

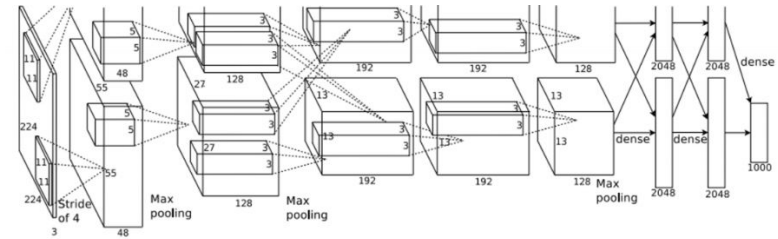
AlexNet



Layer	Input size		Layer				Output size		memory (KB)	params (k)	flop (M)
	C	H / W	filters	kernel	stride	pad	C	H / W			
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	?				

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27			

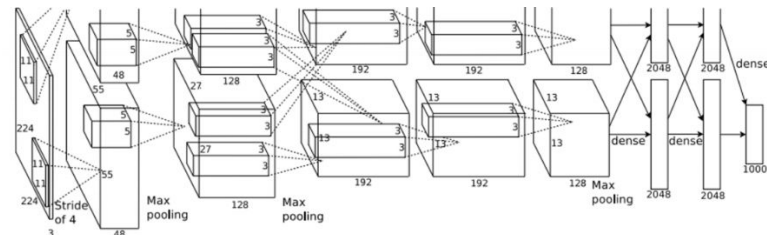
For pooling layer:

#output channels = #input channels = 64

$$\begin{aligned}
 W' &= \text{floor}((W - K) / S + 1) \\
 &= \text{floor}(53 / 2 + 1) = \text{floor}(27.5) = \mathbf{27}
 \end{aligned}$$

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	?	

$$\# \text{output elems} = C_{\text{out}} \times H' \times W'$$

$$\text{Bytes per elem} = 4$$

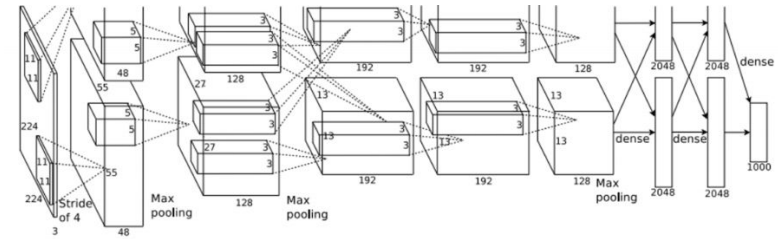
$$\text{KB} = C_{\text{out}} * H' * W' * 4 / 1024$$

$$= 64 * 27 * 27 * 4 / 1024$$

$$= \mathbf{182.25}$$

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet

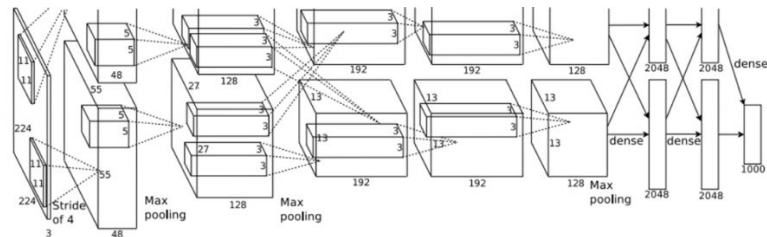


Layer	Input size		Layer				Output size		memory (KB)	params (k)	flop (M)
	C	H / W	filters	kernel	stride	pad	C	H / W			
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	0	?

Pooling layers have no learnable parameters!

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	0	0

Floating-point ops for pooling layer

= (number of output positions) * (flops per output position)

= $(C_{out} * H' * W') * (K * K)$

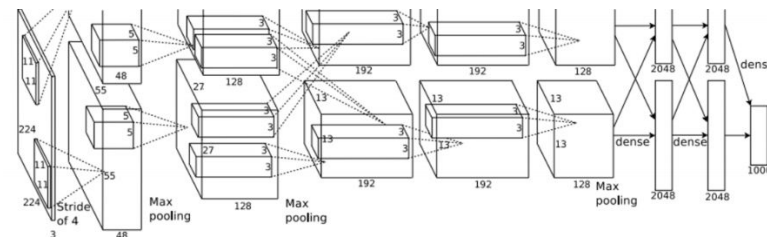
= $(64 * 27 * 27) * (3 * 3)$

= 419,904

= **0.4 MFLOP**

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet

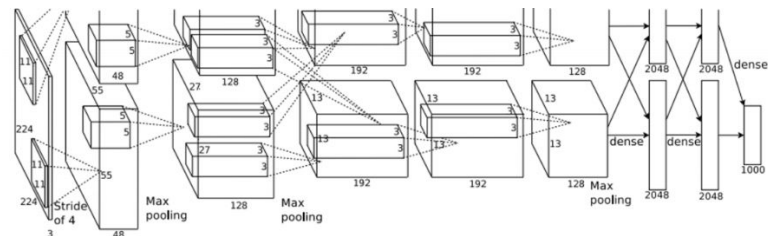


Layer	Input size		Layer				Output size		memory (KB)	params (k)	flop (M)
	C	H / W	filters	kernel	stride	pad	C	H / W			
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	0	0
conv2	64	27	192	5	1	2	192	27	547	307	224
pool2	192	27		3	2	0	192	13	127	0	0
conv3	192	13	384	3	1	1	384	13	254	664	112
conv4	384	13	256	3	1	1	256	13	169	885	145
conv5	256	13	256	3	1	1	256	13	169	590	100
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0

$$\begin{aligned}
 \text{Flatten output size} &= C_{in} \times H \times W \\
 &= 256 * 6 * 6 \\
 &= \mathbf{9216}
 \end{aligned}$$

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet

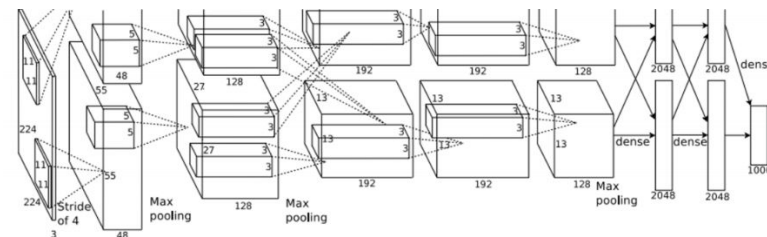


Input size		Layer						Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)	
conv1	3	227	64	11	4	2	64	56	784	23	73	
pool1	64	56		3	2	0	64	27	182	0	0	
conv2	64	27	192	5	1	2	192	27	547	307	224	
pool2	192	27		3	2	0	192	13	127	0	0	
conv3	192	13	384	3	1	1	384	13	254	664	112	
conv4	384	13	256	3	1	1	256	13	169	885	145	
conv5	256	13	256	3	1	1	256	13	169	590	100	
pool5	256	13		3	2	0	256	6	36	0	0	
flatten	256	6					9216		36	0	0	
fc6	9216		4096				4096		16	37,749	38	

$$\begin{aligned}
 \text{FC params} &= C_{\text{in}} * C_{\text{out}} + C_{\text{out}} \\
 &= 9216 * 4096 + 4096 \\
 &= 37,725,832
 \end{aligned}$$

$$\begin{aligned}
 \text{FC flops} &= C_{\text{in}} * C_{\text{out}} \\
 &= 9216 * 4096 \\
 &= 37,748,736
 \end{aligned}$$

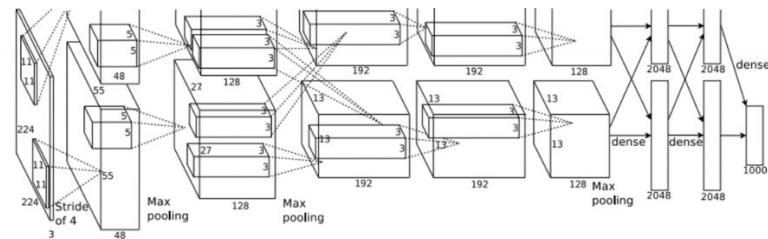
AlexNet



	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	0	0
conv2	64	27	192	5	1	2	192	27	547	307	224
pool2	192	27		3	2	0	192	13	127	0	0
conv3	192	13	384	3	1	1	384	13	254	664	112
conv4	384	13	256	3	1	1	256	13	169	885	145
conv5	256	13	256	3	1	1	256	13	169	590	100
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0
fc6	9216		4096				4096		16	37,749	38
fc7	4096		4096				4096		16	16,777	17
fc8	4096		1000				1000		4	4,096	4

AlexNet

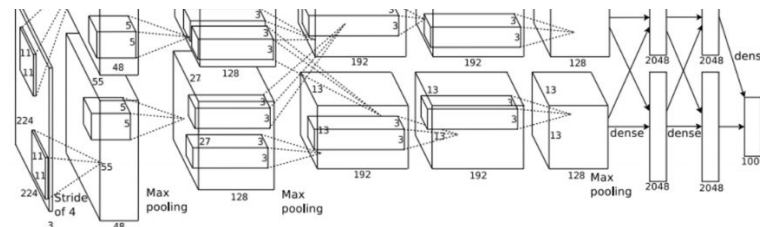
How to choose this?
Trial and error =(



Input size		Layer					Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	0	0
conv2	64	27	192	5	1	2	192	27	547	307	224
pool2	192	27		3	2	0	192	13	127	0	0
conv3	192	13	384	3	1	1	384	13	254	664	112
conv4	384	13	256	3	1	1	256	13	169	885	145
conv5	256	13	256	3	1	1	256	13	169	590	100
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0
fc6	9216		4096				4096		16	37,749	38
fc7	4096		4096				4096		16	16,777	17
fc8	4096		1000				1000		4	4,096	4

AlexNet

Interesting trends here!

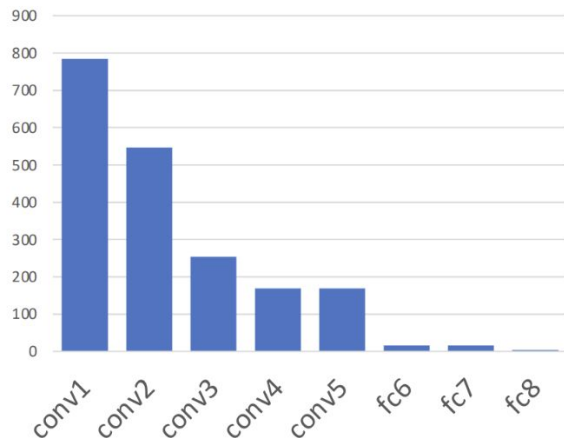


Layer	Input size		Layer				Output size		memory (KB)	params (k)	flop (M)
	C	H / W	filters	kernel	stride	pad	C	H / W			
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	0	0
conv2	64	27	192	5	1	2	192	27	547	307	224
pool2	192	27		3	2	0	192	13	127	0	0
conv3	192	13	384	3	1	1	384	13	254	664	112
conv4	384	13	256	3	1	1	256	13	169	885	145
conv5	256	13	256	3	1	1	256	13	169	590	100
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0
fc6	9216		4096				4096		16	37,749	38
fc7	4096		4096				4096		16	16,777	17
fc8	4096		1000				1000		4	4,096	4

AlexNet

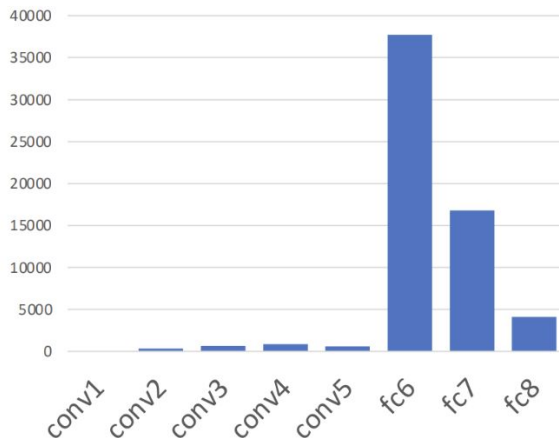
Most of the **memory usage** is in the early convolution layers

Memory (KB)



Nearly all **parameters** are in the fully-connected layers

Params (K)



Most **floating-point ops** occur in the convolution layers

MFLOP

